Internet Engineering Task Force (IETF)
Request for Comments: 7252
Category: Standards Track
ISSN: 2070-1721

Z. Shelby
ARM
K. Hartke
C. Bormann
Universitaet Bremen TZI

June 2014

# The Constrained Application Protocol (CoAP)

## Abstract

The Constrained Application Protocol (CoAP) is a specialized web transfer protocol for use with constrained nodes and constrained (e.g., low-power, lossy) networks. The nodes often have 8-bit microcontrollers with small amounts of ROM and RAM, while constrained networks such as IPv6 over Low-Power Wireless Personal Area Networks (6LoWPANs) often have high packet error rates and a typical throughput of 10s of kbit/s. The protocol is designed for machine-to-machine (M2M) applications such as smart energy and building automation.

CoAP provides a request/response interaction model between application endpoints, supports built-in discovery of services and resources, and includes key concepts of the Web such as URIs and Internet media types. CoAP is designed to easily interface with HTTP for integration with the Web while meeting specialized requirements such as multicast support, very low overhead, and simplicity for constrained environments.

## Status of This Memo

This is an Internet Standards Track document.

This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 5741.

Information about the current status of this document, any errata, and how to provide feedback on it may be obtained at http://www.rfc-editor.org/info/rfc7252.

Copyright Notice

Copyright (c) 2014 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (http://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Simplified BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Simplified BSD License.

Table of Contents

1. Introduction . . . . . 5
1.1. Features . . . . . 5
1.2. Terminology . . . . . 6
2. Constrained Application Protocol . . . . . 10
2.1. Messaging Model . . . . . 11
2.2. Request/Response Model . . . . . 12
2.3. Intermediaries and Caching . . . . . 15
2.4. Resource Discovery . . . . . 15
3. Message Format . . . . . 15
3.1. Option Format . . . . . 17
3.2. Option Value Formats . . . . . 19
4. Message Transmission . . . . . 20
4.1. Messages and Endpoints . . . . . 20
4.2. Messages Transmitted Reliably . . . . . 21
4.3. Messages Transmitted without Reliability . . . . . 23
4.4. Message Correlation . . . . . 24
4.5. Message Deduplication . . . . . 24
4.6. Message Size . . . . . 25
4.7. Congestion Control . . . . . 26
4.8. Transmission Parameters . . . . . 27
4.8.1. Changing the Parameters . . . . . 27
4.8.2. Time Values Derived from Transmission Parameters . . 28
5. Request/Response Semantics . . . . . 31
5.1. Requests . . . . . 31
5.2. Responses . . . . . 31
5.2.1. Piggybacked . . . . . 33
5.2.2. Separate . . . . . 33
5.2.3. Non-confirmable . . . . . 34
5.3. Request/Response Matching . . . . . 34
5.3.1. Token . . . . . 34
5.3.2. Request/Response Matching Rules . . . . . 35

5.4. Options . . . . . 36
5.4.1. Critical/Elective . . . . . 37
5.4.2. Proxy Unsafe or Safe-to-Forward and NoCacheKey . . . 38
5.4.3. Length . . . . . 38
5.4.4. Default Values . . . . . 38
5.4.5. Repeatable Options . . . . . 39
5.4.6. Option Numbers . . . . . 39
5.5. Payloads and Representations . . . . . 40
5.5.1. Representation . . . . . 40
5.5.2. Diagnostic Payload . . . . . 41
5.5.3. Selected Representation . . . . . 41
5.5.4. Content Negotiation . . . . . 41
5.6. Caching . . . . . 42
5.6.1. Freshness Model . . . . . 43
5.6.2. Validation Model . . . . . 43
5.7. Proxying . . . . . 44
5.7.1. Proxy Operation . . . . . 44
5.7.2. Forward-Proxies . . . . . 46
5.7.3. Reverse-Proxies . . . . . 46
5.8. Method Definitions . . . . . 47
5.8.1. GET . . . . . 47
5.8.2. POST . . . . . 47
5.8.3. PUT . . . . . 48
5.8.4. DELETE . . . . . 48
5.9. Response Code Definitions . . . . . 48
5.9.1. Success 2.xx . . . . . 48
5.9.2. Client Error 4.xx . . . . . 50
5.9.3. Server Error 5.xx . . . . . 51
5.10. Option Definitions . . . . . 52
5.10.1. Uri-Host, Uri-Port, Uri-Path, and Uri-Query . . . . 53
5.10.2. Proxy-Uri and Proxy-Scheme . . . . . 54
5.10.3. Content-Format . . . . . 55
5.10.4. Accept . . . . . 55
5.10.5. Max-Age . . . . . 55
5.10.6. ETag . . . . . 56
5.10.7. Location-Path and Location-Query . . . . . 57
5.10.8. Conditional Request Options . . . . . 57
5.10.9. Size1 Option . . . . . 59
6. CoAP URIs . . . . . 59
6.1. coap URI Scheme . . . . . 59
6.2. coaps URI Scheme . . . . . 60
6.3. Normalization and Comparison Rules . . . . . 61
6.4. Decomposing URIs into Options . . . . . 61
6.5. Composing URIs from Options . . . . . 62
7. Discovery . . . . . 64
7.1. Service Discovery . . . . . 64
7.2. Resource Discovery . . . . . 64
7.2.1. 'ct' Attribute . . . . . 64

8. Multicast CoAP . . . . . 65
8.1. Messaging Layer . . . . . 65
8.2. Request/Response Layer . . . . . 66
8.2.1. Caching . . . . . 67
8.2.2. Proxying . . . . . 67
9. Securing CoAP . . . . . 68
9.1. DTLS-Secured CoAP . . . . . 69
9.1.1. Messaging Layer . . . . . 70
9.1.2. Request/Response Layer . . . . . 71
9.1.3. Endpoint Identity . . . . . 71
10. Cross-Protocol Proxying between CoAP and HTTP . . . . . 74
10.1. CoAP-HTTP Proxying . . . . . 75
10.1.1. GET . . . . . 76
10.1.2. PUT . . . . . 77
10.1.3. DELETE . . . . . 77
10.1.4. POST . . . . . 77
10.2. HTTP-CoAP Proxying . . . . . 77
10.2.1. OPTIONS and TRACE . . . . . 78
10.2.2. GET . . . . . 78
10.2.3. HEAD . . . . . 79
10.2.4. POST . . . . . 79
10.2.5. PUT . . . . . 79
10.2.6. DELETE . . . . . 80
10.2.7. CONNECT . . . . . 80
11. Security Considerations . . . . . 80
11.1. Parsing the Protocol and Processing URIs . . . . . 80
11.2. Proxying and Caching . . . . . 81
11.3. Risk of Amplification . . . . . 81
11.4. IP Address Spoofing Attacks . . . . . 83
11.5. Cross-Protocol Attacks . . . . . 84
11.6. Constrained-Node Considerations . . . . . 86
12. IANA Considerations . . . . . 86
12.1. CoAP Code Registries . . . . . 86
12.1.1. Method Codes . . . . . 87
12.1.2. Response Codes . . . . . 88
12.2. CoAP Option Numbers Registry . . . . . 89
12.3. CoAP Content-Formats Registry . . . . . 91
12.4. URI Scheme Registration . . . . . 93
12.5. Secure URI Scheme Registration . . . . . 94
12.6. Service Name and Port Number Registration . . . . . 95
12.7. Secure Service Name and Port Number Registration . . . . 96
12.8. Multicast Address Registration . . . . . 97
13. Acknowledgements . . . . . 97
14. References . . . . . 98
14.1. Normative References . . . . . 98
14.2. Informative References . . . . . 100
Appendix A. Examples . . . . . 104
Appendix B. URI Examples . . . . . 110

# 1. Introduction

The use of web services (web APIs) on the Internet has become ubiquitous in most applications and depends on the fundamental Representational State Transfer [REST] architecture of the Web.

The work on Constrained RESTful Environments (CoRE) aims at realizing the REST architecture in a suitable form for the most constrained nodes (e.g., 8-bit microcontrollers with limited RAM and ROM) and networks (e.g., 6LoWPAN, [RFC4944]). Constrained networks such as 6LoWPAN support the fragmentation of IPv6 packets into small link-layer frames; however, this causes significant reduction in packet delivery probability. One design goal of CoAP has been to keep message overhead small, thus limiting the need for fragmentation.

One of the main goals of CoAP is to design a generic web protocol for the special requirements of this constrained environment, especially considering energy, building automation, and other machine-to-machine (M2M) applications. The goal of CoAP is not to blindly compress HTTP [RFC2616], but rather to realize a subset of REST common with HTTP but optimized for M2M applications. Although CoAP could be used for refashioning simple HTTP interfaces into a more compact protocol, more importantly it also offers features for M2M such as built-in discovery, multicast support, and asynchronous message exchanges.

This document specifies the Constrained Application Protocol (CoAP), which easily translates to HTTP for integration with the existing Web while meeting specialized requirements such as multicast support, very low overhead, and simplicity for constrained environments and M2M applications.

## 1.1. Features

CoAP has the following main features:

- Web protocol fulfilling M2M requirements in constrained environments
- UDP [RFC0768] binding with optional reliability supporting unicast and multicast requests.
- Asynchronous message exchanges.
- Low header overhead and parsing complexity.
- URI and Content-type support.
- Simple proxy and caching capabilities.

- A stateless HTTP mapping, allowing proxies to be built providing access to CoAP resources via HTTP in a uniform way or for HTTP simple interfaces to be realized alternatively over CoAP.

- Security binding to Datagram Transport Layer Security (DTLS) [RFC6347].

## 1.2. Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119] when they appear in ALL CAPS. These words may also appear in this document in lowercase, absent their normative meanings.

This specification requires readers to be familiar with all the terms and concepts that are discussed in [RFC2616], including "resource", "representation", "cache", and "fresh". (Having been completed before the updated set of HTTP RFCs, RFC 7230 to RFC 7235, became available, this specification specifically references the predecessor version -- RFC 2616.) In addition, this specification defines the following terminology:

Endpoint
: An entity participating in the CoAP protocol. Colloquially, an endpoint lives on a "Node", although "Host" would be more consistent with Internet standards usage, and is further identified by transport-layer multiplexing information that can include a UDP port number and a security association (Section 4.1).

Sender
: The originating endpoint of a message. When the aspect of identification of the specific sender is in focus, also "source endpoint".

Recipient
: The destination endpoint of a message. When the aspect of identification of the specific recipient is in focus, also "destination endpoint".

Client
: The originating endpoint of a request; the destination endpoint of a response.

Server
: The destination endpoint of a request; the originating endpoint of a response.

Origin Server
: The server on which a given resource resides or is to be created.

Intermediary
: A CoAP endpoint that acts both as a server and as a client towards an origin server (possibly via further intermediaries). A common form of an intermediary is a proxy; several classes of such proxies are discussed in this specification.

Proxy
: An intermediary that mainly is concerned with forwarding requests and relaying back responses, possibly performing caching, namespace translation, or protocol translation in the process. As opposed to intermediaries in the general sense, proxies generally do not implement specific application semantics. Based on the position in the overall structure of the request forwarding, there are two common forms of proxy: forward-proxy and reverse-proxy. In some cases, a single endpoint might act as an origin server, forward-proxy, or reverse-proxy, switching behavior based on the nature of each request.

Forward-Proxy
: An endpoint selected by a client, usually via local configuration rules, to perform requests on behalf of the client, doing any necessary translations. Some translations are minimal, such as for proxy requests for "coap" URIs, whereas other requests might require translation to and from entirely different application-layer protocols.

Reverse-Proxy
: An endpoint that stands in for one or more other server(s) and satisfies requests on behalf of these, doing any necessary translations. Unlike a forward-proxy, the client may not be aware that it is communicating with a reverse-proxy; a reverse-proxy receives requests as if it were the origin server for the target resource.

CoAP-to-CoAP Proxy
: A proxy that maps from a CoAP request to a CoAP request, i.e., uses the CoAP protocol both on the server and the client side. Contrast to cross-proxy.

Cross-Proxy
: A cross-protocol proxy, or "cross-proxy" for short, is a proxy that translates between different protocols, such as a CoAP-to-HTTP proxy or an HTTP-to-CoAP proxy. While this specification makes very specific demands of CoAP-to-CoAP proxies, there is more variation possible in cross-proxies.

Confirmable Message
: Some messages require an acknowledgement. These messages are called "Confirmable". When no packets are lost, each Confirmable message elicits exactly one return message of type Acknowledgement or type Reset.

Non-confirmable Message
: Some other messages do not require an acknowledgement. This is particularly true for messages that are repeated regularly for application requirements, such as repeated readings from a sensor.

Acknowledgement Message
: An Acknowledgement message acknowledges that a specific Confirmable message arrived. By itself, an Acknowledgement message does not indicate success or failure of any request encapsulated in the Confirmable message, but the Acknowledgement message may also carry a Piggybacked Response (see below).

Reset Message
: A Reset message indicates that a specific message (Confirmable or Non-confirmable) was received, but some context is missing to properly process it. This condition is usually caused when the receiving node has rebooted and has forgotten some state that would be required to interpret the message. Provoking a Reset message (e.g., by sending an Empty Confirmable message) is also useful as an inexpensive check of the liveness of an endpoint ("CoAP ping").

Piggybacked Response
: A piggybacked Response is included right in a CoAP Acknowledgement (ACK) message that is sent to acknowledge receipt of the Request for this Response (Section 5.2.1).

Separate Response
: When a Confirmable message carrying a request is acknowledged with an Empty message (e.g., because the server doesn't have the answer right away), a Separate Response is sent in a separate message exchange (Section 5.2.2).

Empty Message
: A message with a Code of 0.00; neither a request nor a response. An Empty message only contains the 4-byte header.

Critical Option
: An option that would need to be understood by the endpoint ultimately receiving the message in order to properly process the message (Section 5.4.1). Note that the implementation of critical options is, as the name "Option" implies, generally optional: unsupported critical options lead to an error response or summary rejection of the message.

Elective Option
: An option that is intended to be ignored by an endpoint that does not understand it. Processing the message even without understanding the option is acceptable (Section 5.4.1).

Unsafe Option
: An option that would need to be understood by a proxy receiving the message in order to safely forward the message (Section 5.4.2). Not every critical option is an unsafe option.

Safe-to-Forward Option
: An option that is intended to be safe for forwarding by a proxy that does not understand it. Forwarding the message even without understanding the option is acceptable (Section 5.4.2).

Resource Discovery
: The process where a CoAP client queries a server for its list of hosted resources (i.e., links as defined in Section 7).

Content-Format
: The combination of an Internet media type, potentially with specific parameters given, and a content-coding (which is often the identity content-coding), identified by a numeric identifier defined by the "CoAP Content-Formats" registry. When the focus is less on the numeric identifier than on the combination of these characteristics of a resource representation, this is also called "representation format".

Additional terminology for constrained nodes and constrained-node networks can be found in [RFC7228].

In this specification, the term "byte" is used in its now customary sense as a synonym for "octet".

All multi-byte integers in this protocol are interpreted in network byte order.

Where arithmetic is used, this specification uses the notation familiar from the programming language C, except that the operator "**" stands for exponentiation.

## 2. Constrained Application Protocol

The interaction model of CoAP is similar to the client/server model of HTTP. However, machine-to-machine interactions typically result in a CoAP implementation acting in both client and server roles. A CoAP request is equivalent to that of HTTP and is sent by a client to request an action (using a Method Code) on a resource (identified by a URI) on a server. The server then sends a response with a Response Code; this response may include a resource representation.

Unlike HTTP, CoAP deals with these interchanges asynchronously over a datagram-oriented transport such as UDP. This is done logically using a layer of messages that supports optional reliability (with exponential back-off). CoAP defines four types of messages: Confirmable, Non-confirmable, Acknowledgement, Reset. Method Codes and Response Codes included in some of these messages make them carry requests or responses. The basic exchanges of the four types of messages are somewhat orthogonal to the request/response interactions; requests can be carried in Confirmable and Non-confirmable messages, and responses can be carried in these as well as piggybacked in Acknowledgement messages.

One could think of CoAP logically as using a two-layer approach, a CoAP messaging layer used to deal with UDP and the asynchronous nature of the interactions, and the request/response interactions using Method and Response Codes (see Figure 1). CoAP is however a single protocol, with messaging and request/response as just features of the CoAP header.

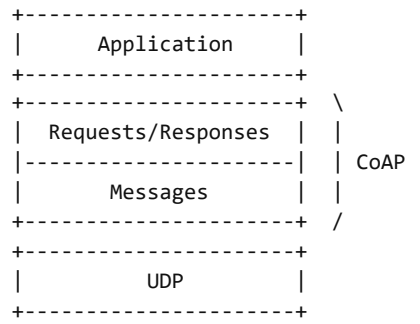



Figure 1: Abstract Layering of CoAP

## 2.1. Messaging Model

The CoAP messaging model is based on the exchange of messages over UDP between endpoints.

CoAP uses a short fixed-length binary header (4 bytes) that may be followed by compact binary options and a payload. This message format is shared by requests and responses. The CoAP message format is specified in Section 3. Each message contains a Message ID used to detect duplicates and for optional reliability. (The Message ID is compact; its 16-bit size enables up to about 250 messages per second from one endpoint to another with default protocol parameters.)

Reliability is provided by marking a message as Confirmable (CON). A Confirmable message is retransmitted using a default timeout and exponential back-off between retransmissions, until the recipient sends an Acknowledgement message (ACK) with the same Message ID (in this example, 0x7d34) from the corresponding endpoint; see Figure 2. When a recipient is not at all able to process a Confirmable message (i.e., not even able to provide a suitable error response), it replies with a Reset message (RST) instead of an Acknowledgement (ACK).

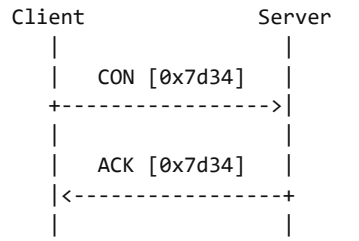



Figure 2: Reliable Message Transmission

A message that does not require reliable transmission (for example, each single measurement out of a stream of sensor data) can be sent as a Non-confirmable message (NON). These are not acknowledged, but still have a Message ID for duplicate detection (in this example, 0x01a0); see Figure 3. When a recipient is not able to process a Non-confirmable message, it may reply with a Reset message (RST).

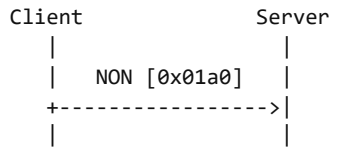



Figure 3: Unreliable Message Transmission

See Section 4 for details of CoAP messages.

As CoAP runs over UDP, it also supports the use of multicast IP destination addresses, enabling multicast CoAP requests. Section 8 discusses the proper use of CoAP messages with multicast addresses and precautions for avoiding response congestion.

Several security modes are defined for CoAP in Section 9 ranging from no security to certificate-based security. This document specifies a binding to DTLS for securing the protocol; the use of IPsec with CoAP is discussed in [IPsec-CoAP].

## 2.2. Request/Response Model

CoAP request and response semantics are carried in CoAP messages, which include either a Method Code or Response Code, respectively. Optional (or default) request and response information, such as the URI and payload media type are carried as CoAP options. A Token is used to match responses to requests independently from the underlying messages (Section 5.3). (Note that the Token is a concept separate from the Message ID.)

A request is carried in a Confirmable (CON) or Non-confirmable (NON) message, and, if immediately available, the response to a request carried in a Confirmable message is carried in the resulting Acknowledgement (ACK) message. This is called a piggybacked response, detailed in Section 5.2.1. (There is no need for separately acknowledging a piggybacked response, as the client will retransmit the request if the Acknowledgement message carrying the piggybacked response is lost.) Two examples for a basic GET request with piggybacked response are shown in Figure 4, one successful, one resulting in a 4.04 (Not Found) response.

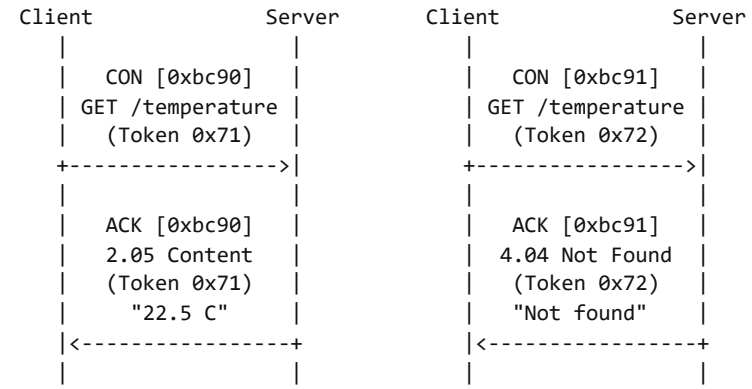



Figure 4: Two GET Requests with Piggybacked Responses

If the server is not able to respond immediately to a request carried in a Confirmable message, it simply responds with an Empty Acknowledgement message so that the client can stop retransmitting the request. When the response is ready, the server sends it in a new Confirmable message (which then in turn needs to be acknowledged by the client). This is called a "separate response", as illustrated in Figure 5 and described in more detail in Section 5.2.2.

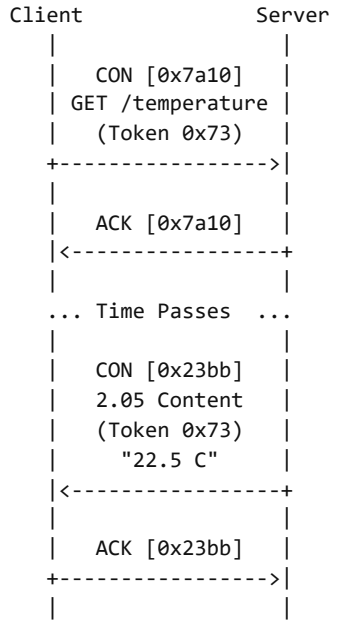



Figure 5: A GET Request with a Separate Response

If a request is sent in a Non-confirmable message, then the response is sent using a new Non-confirmable message, although the server may instead send a Confirmable message. This type of exchange is illustrated in Figure 6.

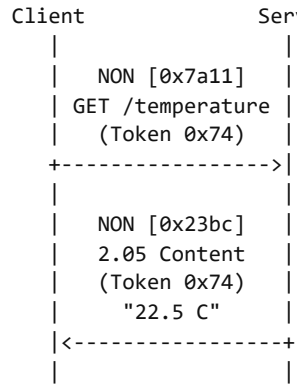



Figure 6: A Request and a Response Carried in Non-confirmable Messages

CoAP makes use of GET, PUT, POST, and DELETE methods in a similar manner to HTTP, with the semantics specified in Section 5.8. (Note that the detailed semantics of CoAP methods are "almost, but not entirely unlike" [HHGTTG] those of HTTP methods: intuition taken from HTTP experience generally does apply well, but there are enough differences that make it worthwhile to actually read the present specification.)

Methods beyond the basic four can be added to CoAP in separate specifications. New methods do not necessarily have to use requests and responses in pairs. Even for existing methods, a single request may yield multiple responses, e.g., for a multicast request (Section 8) or with the Observe option [OBSERVE].

URI support in a server is simplified as the client already parses the URI and splits it into host, port, path, and query components, making use of default values for efficiency. Response Codes relate to a small subset of HTTP status codes with a few CoAP-specific codes added, as defined in Section 5.9.

### 2.3. Intermediaries and Caching

The protocol supports the caching of responses in order to efficiently fulfill requests. Simple caching is enabled using freshness and validity information carried with CoAP responses. A cache could be located in an endpoint or an intermediary. Caching functionality is specified in Section 5.6.

Proxying is useful in constrained networks for several reasons, including to limit network traffic, to improve performance, to access resources of sleeping devices, and for security reasons. The proxying of requests on behalf of another CoAP endpoint is supported in the protocol. When using a proxy, the URI of the resource to request is included in the request, while the destination IP address is set to the address of the proxy. See Section 5.7 for more information on proxy functionality.

As CoAP was designed according to the REST architecture [REST], and thus exhibits functionality similar to that of the HTTP protocol, it is quite straightforward to map from CoAP to HTTP and from HTTP to CoAP. Such a mapping may be used to realize an HTTP REST interface using CoAP or to convert between HTTP and CoAP. This conversion can be carried out by a cross-protocol proxy ("cross-proxy"), which converts the Method or Response Code, media type, and options to the corresponding HTTP feature. Section 10 provides more detail about HTTP mapping.

### 2.4. Resource Discovery

Resource discovery is important for machine-to-machine interactions and is supported using the CoRE Link Format [RFC6690] as discussed in Section 7.

## 3. Message Format

CoAP is based on the exchange of compact messages that, by default, are transported over UDP (i.e., each CoAP message occupies the data section of one UDP datagram). CoAP may also be used over Datagram Transport Layer Security (DTLS) (see Section 9.1). It could also be used over other transports such as SMS, TCP, or SCTP, the specification of which is out of this document's scope. (UDP-lite [RFC3828] and UDP zero checksum [RFC6936] are not supported by CoAP.)

CoAP messages are encoded in a simple binary format. The message format starts with a fixed-size 4-byte header. This is followed by a variable-length Token value, which can be between 0 and 8 bytes long.

Following the Token value comes a sequence of zero or more CoAP Options in Type-Length-Value (TLV) format, optionally followed by a payload that takes up the rest of the datagram.

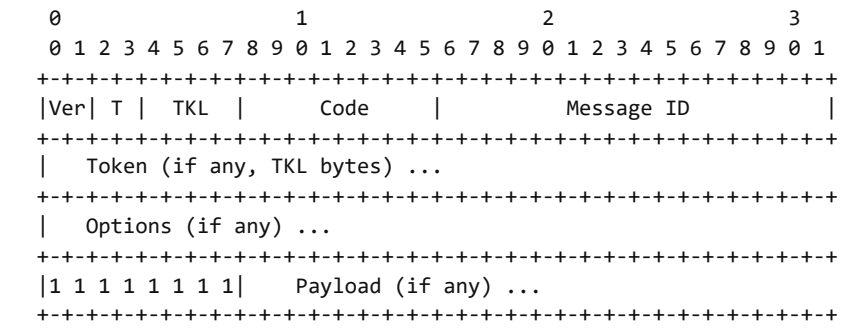



Figure 7: Message Format

The fields in the header are defined as follows:

Version (Ver): 2-bit unsigned integer. Indicates the CoAP version number. Implementations of this specification MUST set this field to 1 (01 binary). Other values are reserved for future versions. Messages with unknown version numbers MUST be silently ignored.

Type (T): 2-bit unsigned integer. Indicates if this message is of type Confirmable (0), Non-confirmable (1), Acknowledgement (2), or Reset (3). The semantics of these message types are defined in Section 4.

Token Length (TKL): 4-bit unsigned integer. Indicates the length of the variable-length Token field (0-8 bytes). Lengths 9-15 are reserved, MUST NOT be sent, and MUST be processed as a message format error.

Code: 8-bit unsigned integer, split into a 3-bit class (most significant bits) and a 5-bit detail (least significant bits), documented as "c.dd" where "c" is a digit from 0 to 7 for the 3-bit subfield and "dd" are two digits from 00 to 31 for the 5-bit subfield. The class can indicate a request (0), a success response (2), a client error response (4), or a server error response (5). (All other class values are reserved.) As a special case, Code 0.00 indicates an Empty message. In case of a request, the Code field indicates the Request Method; in case of a response, a Response Code. Possible values are maintained in the CoAP Code Registries (Section 12.1). The semantics of requests and responses are defined in Section 5.

Message ID: 16-bit unsigned integer in network byte order. Used to detect message duplication and to match messages of type Acknowledgement/Reset to messages of type Confirmable/Non-confirmable. The rules for generating a Message ID and matching messages are defined in Section 4.

The header is followed by the Token value, which may be 0 to 8 bytes, as given by the Token Length field. The Token value is used to correlate requests and responses. The rules for generating a Token and correlating requests and responses are defined in Section 5.3.1.

Header and Token are followed by zero or more Options (Section 3.1). An Option can be followed by the end of the message, by another Option, or by the Payload Marker and the payload.

Following the header, token, and options, if any, comes the optional payload. If present and of non-zero length, it is prefixed by a fixed, one-byte Payload Marker (0xFF), which indicates the end of options and the start of the payload. The payload data extends from after the marker to the end of the UDP datagram, i.e., the Payload Length is calculated from the datagram size. The absence of the Payload Marker denotes a zero-length payload. The presence of a marker followed by a zero-length payload MUST be processed as a message format error.

Implementation Note: The byte value 0xFF may also occur within an option length or value, so simple byte-wise scanning for 0xFF is not a viable technique for finding the payload marker. The byte 0xFF has the meaning of a payload marker only where the beginning of another option could occur.

## 3.1. Option Format

CoAP defines a number of options that can be included in a message. Each option instance in a message specifies the Option Number of the defined CoAP option, the length of the Option Value, and the Option Value itself.

Instead of specifying the Option Number directly, the instances MUST appear in order of their Option Numbers and a delta encoding is used between them: the Option Number for each instance is calculated as the sum of its delta and the Option Number of the preceding instance in the message. For the first instance in a message, a preceding option instance with Option Number zero is assumed. Multiple instances of the same option can be included by using a delta of zero.

Option Numbers are maintained in the "CoAP Option Numbers" registry (Section 12.2). See Section 5.4 for the semantics of the options defined in this document.

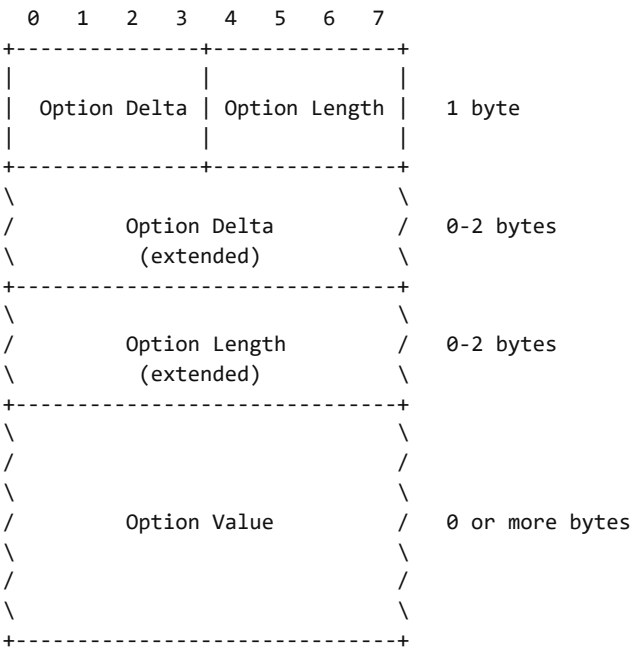



Figure 8: Option Format

The fields in an option are defined as follows:

Option Delta: 4-bit unsigned integer. A value between 0 and 12 indicates the Option Delta. Three values are reserved for special constructs:

13: An 8-bit unsigned integer follows the initial byte and indicates the Option Delta minus 13.

14: A 16-bit unsigned integer in network byte order follows the initial byte and indicates the Option Delta minus 269.

15: Reserved for the Payload Marker. If the field is set to this value but the entire byte is not the payload marker, this MUST be processed as a message format error.

The resulting Option Delta is used as the difference between the Option Number of this option and that of the previous option (or zero for the first option). In other words, the Option Number is calculated by simply summing the Option Delta values of this and all previous options before it.

Option Length: 4-bit unsigned integer. A value between 0 and 12 indicates the length of the Option Value, in bytes. Three values are reserved for special constructs:

13: An 8-bit unsigned integer precedes the Option Value and indicates the Option Length minus 13.

14: A 16-bit unsigned integer in network byte order precedes the Option Value and indicates the Option Length minus 269.

15: Reserved for future use. If the field is set to this value, it MUST be processed as a message format error.

Value: A sequence of exactly Option Length bytes. The length and format of the Option Value depend on the respective option, which MAY define variable-length values. See Section 3.2 for the formats used in this document; options defined in other documents MAY make use of other option value formats.

## 3.2. Option Value Formats

The options defined in this document make use of the following option value formats.

empty: A zero-length sequence of bytes.

opaque: An opaque sequence of bytes.

uint: A non-negative integer that is represented in network byte order using the number of bytes given by the Option Length field.

An option definition may specify a range of permissible numbers of bytes; if it has a choice, a sender SHOULD represent the integer with as few bytes as possible, i.e., without leading zero bytes. For example, the number 0 is represented with an empty option value (a zero-length sequence of bytes) and the number 1 by a single byte with the numerical value of 1 (bit combination 00000001 in most significant bit first notation). A recipient MUST be prepared to process values with leading zero bytes.

Implementation Note: The exceptional behavior permitted for the sender is intended for highly constrained, templated implementations (e.g., hardware implementations) that use fixed-size options in the templates.

string: A Unicode string that is encoded using UTF-8 [RFC3629] in Net-Unicode form [RFC5198].

Note that here, and in all other places where UTF-8 encoding is used in the CoAP protocol, the intention is that the encoded strings can be directly used and compared as opaque byte strings by CoAP protocol implementations. There is no expectation and no need to perform normalization within a CoAP implementation (except where Unicode strings that are not known to be normalized are imported from sources outside the CoAP protocol). Note also that ASCII strings (that do not make use of special control characters) are always valid UTF-8 Net-Unicode strings.

## 4. Message Transmission

CoAP messages are exchanged asynchronously between CoAP endpoints. They are used to transport CoAP requests and responses, the semantics of which are defined in Section 5.

As CoAP is bound to unreliable transports such as UDP, CoAP messages may arrive out of order, appear duplicated, or go missing without notice. For this reason, CoAP implements a lightweight reliability mechanism, without trying to re-create the full feature set of a transport like TCP. It has the following features:

- Simple stop-and-wait retransmission reliability with exponential back-off for Confirmable messages.
- Duplicate detection for both Confirmable and Non-confirmable messages.

### 4.1. Messages and Endpoints

A CoAP endpoint is the source or destination of a CoAP message. The specific definition of an endpoint depends on the transport being used for CoAP. For the transports defined in this specification, the endpoint is identified depending on the security mode used (see Section 9): With no security, the endpoint is solely identified by an IP address and a UDP port number. With other security modes, the endpoint is identified as defined by the security mode.

There are different types of messages. The type of a message is specified by the Type field of the CoAP Header.

Separate from the message type, a message may carry a request, a response, or be Empty. This is signaled by the Request/Response Code field in the CoAP Header and is relevant to the request/response model. Possible values for the field are maintained in the CoAP Code Registries (Section 12.1).

An Empty message has the Code field set to 0.00. The Token Length field MUST be set to 0 and bytes of data MUST NOT be present after the Message ID field. If there are any bytes, they MUST be processed as a message format error.

## 4.2. Messages Transmitted Reliably

The reliable transmission of a message is initiated by marking the message as Confirmable in the CoAP header. A Confirmable message always carries either a request or response, unless it is used only to elicit a Reset message, in which case it is Empty. A recipient MUST either (a) acknowledge a Confirmable message with an Acknowledgement message or (b) reject the message if the recipient lacks context to process the message properly, including situations where the message is Empty, uses a code with a reserved class (1, 6, or 7), or has a message format error. Rejecting a Confirmable message is effected by sending a matching Reset message and otherwise ignoring it. The Acknowledgement message MUST echo the Message ID of the Confirmable message and MUST carry a response or be Empty (see Sections 5.2.1 and 5.2.2). The Reset message MUST echo the Message ID of the Confirmable message and MUST be Empty. Rejecting an Acknowledgement or Reset message (including the case where the Acknowledgement carries a request or a code with a reserved class, or the Reset message is not Empty) is effected by silently ignoring it. More generally, recipients of Acknowledgement and Reset messages MUST NOT respond with either Acknowledgement or Reset messages.

The sender retransmits the Confirmable message at exponentially increasing intervals, until it receives an acknowledgement (or Reset message) or runs out of attempts.

Retransmission is controlled by two things that a CoAP endpoint MUST keep track of for each Confirmable message it sends while waiting for an acknowledgement (or reset): a timeout and a retransmission counter. For a new Confirmable message, the initial timeout is set to a random duration (often not an integral number of seconds) between ACK_TIMEOUT and (ACK_TIMEOUT * ACK_RANDOM_FACTOR) (see Section 4.8), and the retransmission counter is set to 0. When the timeout is triggered and the retransmission counter is less than

MAX_RETRANSMIT, the message is retransmitted, the retransmission counter is incremented, and the timeout is doubled. If the retransmission counter reaches MAX_RETRANSMIT on a timeout, or if the endpoint receives a Reset message, then the attempt to transmit the message is canceled and the application process informed of failure. On the other hand, if the endpoint receives an acknowledgement in time, transmission is considered successful.

This specification makes no strong requirements on the accuracy of the clocks used to implement the above binary exponential back-off algorithm. In particular, an endpoint may be late for a specific retransmission due to its sleep schedule and may catch up on the next one. However, the minimum spacing before another retransmission is ACK_TIMEOUT, and the entire sequence of (re-)transmissions MUST stay in the envelope of MAX_TRANSMIT_SPAN (see Section 4.8.2), even if that means a sender may miss an opportunity to transmit.

A CoAP endpoint that sent a Confirmable message MAY give up in attempting to obtain an ACK even before the MAX_RETRANSMIT counter value is reached. For example, the application has canceled the request as it no longer needs a response, or there is some other indication that the CON message did arrive. In particular, a CoAP request message may have elicited a separate response, in which case it is clear to the requester that only the ACK was lost and a retransmission of the request would serve no purpose. However, a responder MUST NOT in turn rely on this cross-layer behavior from a requester, i.e., it MUST retain the state to create the ACK for the request, if needed, even if a Confirmable response was already acknowledged by the requester.

Another reason for giving up retransmission MAY be the receipt of ICMP errors. If it is desired to take account of ICMP errors, to mitigate potential spoofing attacks, implementations SHOULD take care to check the information about the original datagram in the ICMP message, including port numbers and CoAP header information such as message type and code, Message ID, and Token; if this is not possible due to limitations of the UDP service API, ICMP errors SHOULD be ignored. Packet Too Big errors [RFC4443] ("fragmentation needed and DF set" for IPv4 [RFC0792]) cannot properly occur and SHOULD be ignored if the implementation note in Section 4.6 is followed; otherwise, they SHOULD feed into a path MTU discovery algorithm [RFC4821]. Source Quench and Time Exceeded ICMP messages SHOULD be ignored. Host, network, port, or protocol unreachable errors or parameter problem errors MAY, after appropriate vetting, be used to inform the application of a failure in sending.

### 4.3. Messages Transmitted without Reliability

Some messages do not require an acknowledgement. This is particularly true for messages that are repeated regularly for application requirements, such as repeated readings from a sensor where eventual success is sufficient.

As a more lightweight alternative, a message can be transmitted less reliably by marking the message as Non-confirmable. A Non-confirmable message always carries either a request or response and MUST NOT be Empty. A Non-confirmable message MUST NOT be acknowledged by the recipient. A recipient MUST reject the message if it lacks context to process the message properly, including the case where the message is Empty, uses a code with a reserved class (1, 6, or 7), or has a message format error. Rejecting a Non-confirmable message MAY involve sending a matching Reset message, and apart from the Reset message the rejected message MUST be silently ignored.

At the CoAP level, there is no way for the sender to detect if a Non-confirmable message was received or not. A sender MAY choose to transmit multiple copies of a Non-confirmable message within MAX_TRANSMIT_SPAN (limited by the provisions of Section 4.7, in particular, by PROBING_RATE if no response is received), or the network may duplicate the message in transit. To enable the receiver to act only once on the message, Non-confirmable messages specify a Message ID as well. (This Message ID is drawn from the same number space as the Message IDs for Confirmable messages.)

Summarizing Sections 4.2 and 4.3, the four message types can be used as in Table 1. "*" means that the combination is not used in normal operation but only to elicit a Reset message ("CoAP ping").

| | CON | NON | ACK | RST |
|---|---|---|---|---|
| Request | X | X | - | - |
| Response | X | X | X | - |
| Empty | * | - | X | X |

Table 1: Usage of Message Types

### 4.4. Message Correlation

An Acknowledgement or Reset message is related to a Confirmable message or Non-confirmable message by means of a Message ID along with additional address information of the corresponding endpoint. The Message ID is a 16-bit unsigned integer that is generated by the sender of a Confirmable or Non-confirmable message and included in the CoAP header. The Message ID MUST be echoed in the Acknowledgement or Reset message by the recipient.

The same Message ID MUST NOT be reused (in communicating with the same endpoint) within the EXCHANGE_LIFETIME (Section 4.8.2).

Implementation Note: Several implementation strategies can be employed for generating Message IDs. In the simplest case, a CoAP endpoint generates Message IDs by keeping a single Message ID variable, which is changed each time a new Confirmable or Non-confirmable message is sent, regardless of the destination address or port. Endpoints dealing with large numbers of transactions could keep multiple Message ID variables, for example, per prefix or destination address. (Note that some receiving endpoints may not be able to distinguish unicast and multicast packets addressed to it, so endpoints generating Message IDs need to make sure these do not overlap.) It is strongly recommended that the initial value of the variable (e.g., on startup) be randomized, in order to make successful off-path attacks on the protocol less likely.

For an Acknowledgement or Reset message to match a Confirmable or Non-confirmable message, the Message ID and source endpoint of the Acknowledgement or Reset message MUST match the Message ID and destination endpoint of the Confirmable or Non-confirmable message.

### 4.5. Message Deduplication

A recipient might receive the same Confirmable message (as indicated by the Message ID and source endpoint) multiple times within the EXCHANGE_LIFETIME (Section 4.8.2), for example, when its Acknowledgement went missing or didn't reach the original sender before the first timeout. The recipient SHOULD acknowledge each duplicate copy of a Confirmable message using the same Acknowledgement or Reset message but SHOULD process any request or response in the message only once. This rule MAY be relaxed in case the Confirmable message transports a request that is idempotent (see Section 5.1) or can be handled in an idempotent fashion. Examples for relaxed message deduplication:

- A server might relax the requirement to answer all retransmissions of an idempotent request with the same response (Section 4.2), so that it does not have to maintain state for Message IDs. For example, an implementation might want to process duplicate transmissions of a GET, PUT, or DELETE request as separate requests if the effort incurred by duplicate processing is less expensive than keeping track of previous responses would be.

- A constrained server might even want to relax this requirement for certain non-idempotent requests if the application semantics make this trade-off favorable. For example, if the result of a POST request is just the creation of some short-lived state at the server, it may be less expensive to incur this effort multiple times for a request than keeping track of whether a previous transmission of the same request already was processed.

A recipient might receive the same Non-confirmable message (as indicated by the Message ID and source endpoint) multiple times within NON_LIFETIME (Section 4.8.2). As a general rule that MAY be relaxed based on the specific semantics of a message, the recipient SHOULD silently ignore any duplicated Non-confirmable message and SHOULD process any request or response in the message only once.

## 4.6. Message Size

While specific link layers make it beneficial to keep CoAP messages small enough to fit into their link-layer packets (see Section 1), this is a matter of implementation quality. The CoAP specification itself provides only an upper bound to the message size. Messages larger than an IP packet result in undesirable packet fragmentation. A CoAP message, appropriately encapsulated, SHOULD fit within a single IP packet (i.e., avoid IP fragmentation) and (by fitting into one UDP payload) obviously needs to fit within a single IP datagram. If the Path MTU is not known for a destination, an IP MTU of 1280 bytes SHOULD be assumed; if nothing is known about the size of the headers, good upper bounds are 1152 bytes for the message size and 1024 bytes for the payload size.

Implementation Note: CoAP's choice of message size parameters works well with IPv6 and with most of today's IPv4 paths. (However, with IPv4, it is harder to absolutely ensure that there is no IP fragmentation. If IPv4 support on unusual networks is a consideration, implementations may want to limit themselves to more conservative IPv4 datagram sizes such as 576 bytes; per [RFC0791], the absolute minimum value of the IP MTU for IPv4 is as low as 68 bytes, which would leave only 40 bytes minus security overhead for a UDP payload. Implementations extremely focused on this problem set might also set the IPv4 DF bit and perform some

form of path MTU discovery [RFC4821]; this should generally be unnecessary in realistic use cases for CoAP, however.) A more important kind of fragmentation in many constrained networks is that on the adaptation layer (e.g., 6LoWPAN L2 packets are limited to 127 bytes including various overheads); this may motivate implementations to be frugal in their packet sizes and to move to block-wise transfers [BLOCK] when approaching three-digit message sizes.

Message sizes are also of considerable importance to implementations on constrained nodes. Many implementations will need to allocate a buffer for incoming messages. If an implementation is too constrained to allow for allocating the above-mentioned upper bound, it could apply the following implementation strategy for messages not using DTLS security: Implementations receiving a datagram into a buffer that is too small are usually able to determine if the trailing portion of a datagram was discarded and to retrieve the initial portion. So, at least the CoAP header and options, if not all of the payload, are likely to fit within the buffer. A server can thus fully interpret a request and return a 4.13 (Request Entity Too Large; see Section 5.9.2.9) Response Code if the payload was truncated. A client sending an idempotent request and receiving a response larger than would fit in the buffer can repeat the request with a suitable value for the Block Option [BLOCK].

## 4.7. Congestion Control

Basic congestion control for CoAP is provided by the exponential back-off mechanism in Section 4.2.

In order not to cause congestion, clients (including proxies) MUST strictly limit the number of simultaneous outstanding interactions that they maintain to a given server (including proxies) to NSTART. An outstanding interaction is either a CON for which an ACK has not yet been received but is still expected (message layer) or a request for which neither a response nor an Acknowledgment message has yet been received but is still expected (which may both occur at the same time, counting as one outstanding interaction). The default value of NSTART for this specification is 1.

Further congestion control optimizations and considerations are expected in the future, may for example provide automatic initialization of the CoAP transmission parameters defined in Section 4.8, and thus may allow a value for NSTART greater than one.

After EXCHANGE_LIFETIME, a client stops expecting a response to a Confirmable request for which no acknowledgment message was received.

The specific algorithm by which a client stops to "expect" a response to a Confirmable request that was acknowledged, or to a Non-confirmable request, is not defined. Unless this is modified by additional congestion control optimizations, it MUST be chosen in such a way that an endpoint does not exceed an average data rate of PROBING_RATE in sending to another endpoint that does not respond.

Note: CoAP places the onus of congestion control mostly on the clients. However, clients may malfunction or actually be attackers, e.g., to perform amplification attacks (Section 11.3). To limit the damage (to the network and to its own energy resources), a server SHOULD implement some rate limiting for its response transmission based on reasonable assumptions about application requirements. This is most helpful if the rate limit can be made effective for the misbehaving endpoints, only.

### 4.8. Transmission Parameters

Message transmission is controlled by the following parameters:

| name | default value |
|---|---|
| ACK_TIMEOUT | 2 seconds |
| ACK_RANDOM_FACTOR | 1.5 |
| MAX_RETRANSMIT | 4 |
| NSTART | 1 |
| DEFAULT_LEISURE | 5 seconds |
| PROBING_RATE | 1 byte/second |

Table 2: CoAP Protocol Parameters

#### 4.8.1. Changing the Parameters

The values for ACK_TIMEOUT, ACK_RANDOM_FACTOR, MAX_RETRANSMIT, NSTART, DEFAULT_LEISURE (Section 8.2), and PROBING_RATE may be configured to values specific to the application environment (including dynamically adjusted values); however, the configuration method is out of scope of this document. It is RECOMMENDED that an application environment use consistent values for these parameters; the specific effects of operating with inconsistent values in an application environment are outside the scope of the present specification.

The transmission parameters have been chosen to achieve a behavior in the presence of congestion that is safe in the Internet. If a configuration desires to use different values, the onus is on the

configuration to ensure these congestion control properties are not violated. In particular, a decrease of ACK_TIMEOUT below 1 second would violate the guidelines of [RFC5405]. ([RTO-CONSIDER] provides some additional background.) CoAP was designed to enable implementations that do not maintain round-trip-time (RTT) measurements. However, where it is desired to decrease the ACK_TIMEOUT significantly or increase NSTART, this can only be done safely when maintaining such measurements. Configurations MUST NOT decrease ACK_TIMEOUT or increase NSTART without using mechanisms that ensure congestion control safety, either defined in the configuration or in future standards documents.

ACK_RANDOM_FACTOR MUST NOT be decreased below 1.0, and it SHOULD have a value that is sufficiently different from 1.0 to provide some protection from synchronization effects.

MAX_RETRANSMIT can be freely adjusted, but a value that is too small will reduce the probability that a Confirmable message is actually received, while a larger value than given here will require further adjustments in the time values (see Section 4.8.2).

If the choice of transmission parameters leads to an increase of derived time values (see Section 4.8.2), the configuration mechanism MUST ensure the adjusted value is also available to all the endpoints with which these adjusted values are to be used to communicate.

### 4.8.2. Time Values Derived from Transmission Parameters

The combination of ACK_TIMEOUT, ACK_RANDOM_FACTOR, and MAX_RETRANSMIT influences the timing of retransmissions, which in turn influences how long certain information items need to be kept by an implementation. To be able to unambiguously reference these derived time values, we give them names as follows:

- MAX_TRANSMIT_SPAN is the maximum time from the first transmission of a Confirmable message to its last retransmission. For the default transmission parameters, the value is (2+4+8+16)*1.5 = 45 seconds, or more generally:

```
ACK_TIMEOUT * ((2 ** MAX_RETRANSMIT) - 1) * ACK_RANDOM_FACTOR
```

- MAX_TRANSMIT_WAIT is the maximum time from the first transmission of a Confirmable message to the time when the sender gives up on receiving an acknowledgement or reset. For the default transmission parameters, the value is (2+4+8+16+32)*1.5 = 93 seconds, or more generally:

```
ACK_TIMEOUT * ((2 ** (MAX_RETRANSMIT + 1)) - 1) *
ACK_RANDOM_FACTOR
```

In addition, some assumptions need to be made on the characteristics of the network and the nodes.

- MAX_LATENCY is the maximum time a datagram is expected to take from the start of its transmission to the completion of its reception. This constant is related to the MSL (Maximum Segment Lifetime) of [RFC0793], which is "arbitrarily defined to be 2 minutes" ([RFC0793] glossary, page 81). Note that this is not necessarily smaller than MAX_TRANSMIT_WAIT, as MAX_LATENCY is not intended to describe a situation when the protocol works well, but the worst-case situation against which the protocol has to guard. We, also arbitrarily, define MAX_LATENCY to be 100 seconds. Apart from being reasonably realistic for the bulk of configurations as well as close to the historic choice for TCP, this value also allows Message ID lifetime timers to be represented in 8 bits (when measured in seconds). In these calculations, there is no assumption that the direction of the transmission is irrelevant (i.e., that the network is symmetric); there is just the assumption that the same value can reasonably be used as a maximum value for both directions. If that is not the case, the following calculations become only slightly more complex.

- PROCESSING_DELAY is the time a node takes to turn around a Confirmable message into an acknowledgement. We assume the node will attempt to send an ACK before having the sender time out, so as a conservative assumption we set it equal to ACK_TIMEOUT.

- MAX_RTT is the maximum round-trip time, or:

```
(2 * MAX_LATENCY) + PROCESSING_DELAY
```

From these values, we can derive the following values relevant to the protocol operation:

- EXCHANGE_LIFETIME is the time from starting to send a Confirmable message to the time when an acknowledgement is no longer expected, i.e., message-layer information about the message exchange can be purged. EXCHANGE_LIFETIME includes a MAX_TRANSMIT_SPAN, a MAX_LATENCY forward, PROCESSING_DELAY, and a MAX_LATENCY for the

way back. Note that there is no need to consider MAX_TRANSMIT_WAIT if the configuration is chosen such that the last waiting period (ACK_TIMEOUT * (2 ** MAX_RETRANSMIT) or the difference between MAX_TRANSMIT_SPAN and MAX_TRANSMIT_WAIT) is less than MAX_LATENCY -- which is a likely choice, as MAX_LATENCY is a worst-case value unlikely to be met in the real world. In this case, EXCHANGE_LIFETIME simplifies to:

MAX_TRANSMIT_SPAN + (2 * MAX_LATENCY) + PROCESSING_DELAY

or 247 seconds with the default transmission parameters.

- NON_LIFETIME is the time from sending a Non-confirmable message to the time its Message ID can be safely reused. If multiple transmission of a NON message is not used, its value is MAX_LATENCY, or 100 seconds. However, a CoAP sender might send a NON message multiple times, in particular for multicast applications. While the period of reuse is not bounded by the specification, an expectation of reliable detection of duplication at the receiver is on the timescales of MAX_TRANSMIT_SPAN. Therefore, for this purpose, it is safer to use the value:

  MAX_TRANSMIT_SPAN + MAX_LATENCY

  or 145 seconds with the default transmission parameters; however, an implementation that just wants to use a single timeout value for retiring Message IDs can safely use the larger value for EXCHANGE_LIFETIME.

Table 3 lists the derived parameters introduced in this subsection with their default values.

| name | default value |
|---|---|
| MAX_TRANSMIT_SPAN | 45 s |
| MAX_TRANSMIT_WAIT | 93 s |
| MAX_LATENCY | 100 s |
| PROCESSING_DELAY | 2 s |
| MAX_RTT | 202 s |
| EXCHANGE_LIFETIME | 247 s |
| NON_LIFETIME | 145 s |

Table 3: Derived Protocol Parameters

# 5. Request/Response Semantics

CoAP operates under a similar request/response model as HTTP: a CoAP endpoint in the role of a "client" sends one or more CoAP requests to a "server", which services the requests by sending CoAP responses. Unlike HTTP, requests and responses are not sent over a previously established connection but are exchanged asynchronously over CoAP messages.

## 5.1. Requests

A CoAP request consists of the method to be applied to the resource, the identifier of the resource, a payload and Internet media type (if any), and optional metadata about the request.

CoAP supports the basic methods of GET, POST, PUT, and DELETE, which are easily mapped to HTTP. They have the same properties of safe (only retrieval) and idempotent (you can invoke it multiple times with the same effects) as HTTP (see Section 9.1 of [RFC2616]). The GET method is safe; therefore, it MUST NOT take any other action on a resource other than retrieval. The GET, PUT, and DELETE methods MUST be performed in such a way that they are idempotent. POST is not idempotent, because its effect is determined by the origin server and dependent on the target resource; it usually results in a new resource being created or the target resource being updated.

A request is initiated by setting the Code field in the CoAP header of a Confirmable or a Non-confirmable message to a Method Code and including request information.

The methods used in requests are described in detail in Section 5.8.

## 5.2. Responses

After receiving and interpreting a request, a server responds with a CoAP response that is matched to the request by means of a client-generated token (Section 5.3); note that this is different from the Message ID that matches a Confirmable message to its Acknowledgement.

A response is identified by the Code field in the CoAP header being set to a Response Code. Similar to the HTTP Status Code, the CoAP Response Code indicates the result of the attempt to understand and satisfy the request. These codes are fully defined in Section 5.9. The Response Code numbers to be set in the Code field of the CoAP header are maintained in the CoAP Response Code Registry (Section 12.1.2).

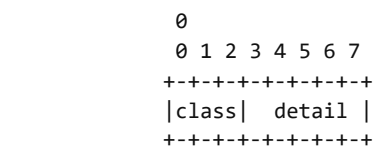



Figure 9: Structure of a Response Code

The upper three bits of the 8-bit Response Code number define the class of response. The lower five bits do not have any categorization role; they give additional detail to the overall class (Figure 9).

As a human-readable notation for specifications and protocol diagnostics, CoAP code numbers including the Response Code are documented in the format "c.dd", where "c" is the class in decimal, and "dd" is the detail as a two-digit decimal. For example, "Forbidden" is written as 4.03 -- indicating an 8-bit code value of hexadecimal 0x83 (4*0x20+3) or decimal 131 (4*32+3).

There are 3 classes of Response Codes:

2 - Success: The request was successfully received, understood, and accepted.

4 - Client Error: The request contains bad syntax or cannot be fulfilled.

5 - Server Error: The server failed to fulfill an apparently valid request.

The Response Codes are designed to be extensible: Response Codes in the Client Error or Server Error class that are unrecognized by an endpoint are treated as being equivalent to the generic Response Code of that class (4.00 and 5.00, respectively). However, there is no generic Response Code indicating success, so a Response Code in the Success class that is unrecognized by an endpoint can only be used to determine that the request was successful without any further details.

The possible Response Codes are described in detail in Section 5.9.

Responses can be sent in multiple ways, which are defined in the following subsections.

#### 5.2.1. Piggybacked

In the most basic case, the response is carried directly in the Acknowledgement message that acknowledges the request (which requires that the request was carried in a Confirmable message). This is called a "Piggybacked Response".

The response is returned in the Acknowledgement message, independent of whether the response indicates success or failure. In effect, the response is piggybacked on the Acknowledgement message, and no separate message is required to return the response.

Implementation Note: The protocol leaves the decision whether to piggyback a response or not (i.e., send a separate response) to the server. The client MUST be prepared to receive either. On the quality-of-implementation level, there is a strong expectation that servers will implement code to piggyback whenever possible -- saving resources in the network and both at the client and at the server.

#### 5.2.2. Separate

It may not be possible to return a piggybacked response in all cases. For example, a server might need longer to obtain the representation of the resource requested than it can wait to send back the Acknowledgement message, without risking the client repeatedly retransmitting the request message (see also the discussion of PROCESSING_DELAY in Section 4.8.2). The response to a request carried in a Non-confirmable message is always sent separately (as there is no Acknowledgement message).

One way to implement this in a server is to initiate the attempt to obtain the resource representation and, while that is in progress, time out an acknowledgement timer. A server may also immediately send an acknowledgement if it knows in advance that there will be no piggybacked response. In both cases, the acknowledgement effectively is a promise that the request will be acted upon later.

When the server finally has obtained the resource representation, it sends the response. When it is desired that this message is not lost, it is sent as a Confirmable message from the server to the client and answered by the client with an Acknowledgement, echoing the new Message ID chosen by the server. (It may also be sent as a Non-confirmable message; see Section 5.2.3.)

When the server chooses to use a separate response, it sends the Acknowledgement to the Confirmable request as an Empty message. Once the server sends back an Empty Acknowledgement, it MUST NOT send back

the response in another Acknowledgement, even if the client retransmits another identical request. If a retransmitted request is received (perhaps because the original Acknowledgement was delayed), another Empty Acknowledgement is sent, and any response MUST be sent as a separate response.

If the server then sends a Confirmable response, the client's Acknowledgement to that response MUST also be an Empty message (one that carries neither a request nor a response). The server MUST stop retransmitting its response on any matching Acknowledgement (silently ignoring any Response Code or payload) or Reset message.

Implementation Notes: Note that, as the underlying datagram transport may not be sequence-preserving, the Confirmable message carrying the response may actually arrive before or after the Acknowledgement message for the request; for the purposes of terminating the retransmission sequence, this also serves as an acknowledgement. Note also that, while the CoAP protocol itself does not make any specific demands here, there is an expectation that the response will come within a time frame that is reasonable from an application point of view. As there is no underlying transport protocol that could be instructed to run a keep-alive mechanism, the requester may want to set up a timeout that is unrelated to CoAP's retransmission timers in case the server is destroyed or otherwise unable to send the response.

#### 5.2.3. Non-confirmable

If the request message is Non-confirmable, then the response SHOULD be returned in a Non-confirmable message as well. However, an endpoint MUST be prepared to receive a Non-confirmable response (preceded or followed by an Empty Acknowledgement message) in reply to a Confirmable request, or a Confirmable response in reply to a Non-confirmable request.

### 5.3. Request/Response Matching

Regardless of how a response is sent, it is matched to the request by means of a token that is included by the client in the request, along with additional address information of the corresponding endpoint.

#### 5.3.1. Token

The Token is used to match a response with a request. The token value is a sequence of 0 to 8 bytes. (Note that every message carries a token, even if it is of zero length.) Every request carries a client-generated token that the server MUST echo (without modification) in any resulting response.

A token is intended for use as a client-local identifier for differentiating between concurrent requests (see Section 5.3); it could have been called a "request ID".

The client SHOULD generate tokens in such a way that tokens currently in use for a given source/destination endpoint pair are unique. (Note that a client implementation can use the same token for any request if it uses a different endpoint each time, e.g., a different source port number.) An empty token value is appropriate e.g., when no other tokens are in use to a destination, or when requests are made serially per destination and receive piggybacked responses. There are, however, multiple possible implementation strategies to fulfill this.

A client sending a request without using Transport Layer Security (Section 9) SHOULD use a nontrivial, randomized token to guard against spoofing of responses (Section 11.4). This protective use of tokens is the reason they are allowed to be up to 8 bytes in size. The actual size of the random component to be used for the Token depends on the security requirements of the client and the level of threat posed by spoofing of responses. A client that is connected to the general Internet SHOULD use at least 32 bits of randomness, keeping in mind that not being directly connected to the Internet is not necessarily sufficient protection against spoofing. (Note that the Message ID adds little in protection as it is usually sequentially assigned, i.e., guessable, and can be circumvented by spoofing a separate response.) Clients that want to optimize the Token length may further want to detect the level of ongoing attacks (e.g., by tallying recent Token mismatches in incoming messages) and adjust the Token length upwards appropriately. [RFC4086] discusses randomness requirements for security.

An endpoint receiving a token it did not generate MUST treat the token as opaque and make no assumptions about its content or structure.

#### 5.3.2. Request/Response Matching Rules

The exact rules for matching a response to a request are as follows:

1. The source endpoint of the response MUST be the same as the destination endpoint of the original request.

2. In a piggybacked response, the Message ID of the Confirmable request and the Acknowledgement MUST match, and the tokens of the response and original request MUST match. In a separate response, just the tokens of the response and original request MUST match.

In case a message carrying a response is unexpected (the client is not waiting for a response from the identified endpoint, at the endpoint addressed, and/or with the given token), the response is rejected (Sections 4.2 and 4.3).

Implementation Note: A client that receives a response in a CON message may want to clean up the message state right after sending the ACK. If that ACK is lost and the server retransmits the CON, the client may no longer have any state to which to correlate this response, making the retransmission an unexpected message; the client will likely send a Reset message so it does not receive any more retransmissions. This behavior is normal and not an indication of an error. (Clients that are not aggressively optimized in their state memory usage will still have message state that will identify the second CON as a retransmission. Clients that actually expect more messages from the server [OBSERVE] will have to keep state in any case.)

### 5.4. Options

Both requests and responses may include a list of one or more options. For example, the URI in a request is transported in several options, and metadata that would be carried in an HTTP header in HTTP is supplied as options as well.

CoAP defines a single set of options that are used in both requests and responses:

- Content-Format
- ETag
- Location-Path
- Location-Query
- Max-Age
- Proxy-Uri
- Proxy-Scheme
- Uri-Host
- Uri-Path
- Uri-Port

- Uri-Query
- Accept
- If-Match
- If-None-Match
- Size1

The semantics of these options along with their properties are defined in detail in Section 5.10.

Not all options are defined for use with all methods and Response Codes. The possible options for methods and Response Codes are defined in Sections 5.8 and 5.9, respectively. In case an option is not defined for a Method or Response Code, it MUST NOT be included by a sender and MUST be treated like an unrecognized option by a recipient.

### 5.4.1. Critical/Elective

Options fall into one of two classes: "critical" or "elective". The difference between these is how an option unrecognized by an endpoint is handled:

- Upon reception, unrecognized options of class "elective" MUST be silently ignored.
- Unrecognized options of class "critical" that occur in a Confirmable request MUST cause the return of a 4.02 (Bad Option) response. This response SHOULD include a diagnostic payload describing the unrecognized option(s) (see Section 5.5.2).
- Unrecognized options of class "critical" that occur in a Confirmable response, or piggybacked in an Acknowledgement, MUST cause the response to be rejected (Section 4.2).
- Unrecognized options of class "critical" that occur in a Non-confirmable message MUST cause the message to be rejected (Section 4.3).

Note that, whether critical or elective, an option is never "mandatory" (it is always optional): these rules are defined in order to enable implementations to stop processing options they do not understand or implement.

Critical/elective rules apply to non-proxying endpoints. A proxy processes options based on Unsafe/Safe-to-Forward classes as defined in Section 5.7.

#### 5.4.2. Proxy Unsafe or Safe-to-Forward and NoCacheKey

In addition to an option being marked as critical or elective, options are also classified based on how a proxy is to deal with the option if it does not recognize it. For this purpose, an option can either be considered Unsafe to forward (UnSafe is set) or Safe-to-Forward (UnSafe is clear).

In addition, for an option that is marked Safe-to-Forward, the option number indicates whether or not it is intended to be part of the Cache-Key (Section 5.6) in a request. If some of the NoCacheKey bits are 0, it is; if all NoCacheKey bits are 1, it is not (see Section 5.4.6).

Note: The Cache-Key indication is relevant only for proxies that do not implement the given option as a request option and instead rely on the Unsafe/Safe-to-Forward indication only. For example, for ETag, actually using the request option as a part of the Cache-Key is grossly inefficient, but it is the best thing one can do if ETag is not implemented by a proxy, as the response is going to differ based on the presence of the request option. A more useful proxy that does implement the ETag request option is not using ETag as a part of the Cache-Key.

NoCacheKey is indicated in three bits so that only one out of eight codepoints is qualified as NoCacheKey, leaving seven out of eight codepoints for what appears to be the more likely case.

Proxy behavior with regard to these classes is defined in Section 5.7.

#### 5.4.3. Length

Option values are defined to have a specific length, often in the form of an upper and lower bound. If the length of an option value in a request is outside the defined range, that option MUST be treated like an unrecognized option (see Section 5.4.1).

#### 5.4.4. Default Values

Options may be defined to have a default value. If the value of an option is intended to be this default value, the option SHOULD NOT be included in the message. If the option is not present, the default value MUST be assumed.

Where a critical option has a default value, this is chosen in such a way that the absence of the option in a message can be processed properly both by implementations unaware of the critical option and by implementations that interpret this absence as the presence of the default value for the option.

### 5.4.5. Repeatable Options

The definition of some options specifies that those options are repeatable. An option that is repeatable MAY be included one or more times in a message. An option that is not repeatable MUST NOT be included more than once in a message.

If a message includes an option with more occurrences than the option is defined for, each supernumerary option occurrence that appears subsequently in the message MUST be treated like an unrecognized option (see Section 5.4.1).

### 5.4.6. Option Numbers

An Option is identified by an option number, which also provides some additional semantics information, e.g., odd numbers indicate a critical option, while even numbers indicate an elective option. Note that this is not just a convention, it is a feature of the protocol: Whether an option is elective or critical is entirely determined by whether its option number is even or odd.

More generally speaking, an Option number is constructed with a bit mask to indicate if an option is Critical or Elective, Unsafe or Safe-to-Forward, and, in the case of Safe-to-Forward, to provide a Cache-Key indication as shown by the following figure. In the following text, the bit mask is expressed as a single byte that is applied to the least significant byte of the option number in unsigned integer representation. When bit 7 (the least significant bit) is 1, an option is Critical (and likewise Elective when 0). When bit 6 is 1, an option is Unsafe (and likewise Safe-to-Forward when 0). When bit 6 is 0, i.e., the option is not Unsafe, it is not a Cache-Key (NoCacheKey) if and only if bits 3-5 are all set to 1; all other bit combinations mean that it indeed is a Cache-Key. These classes of options are explained in the next sections.

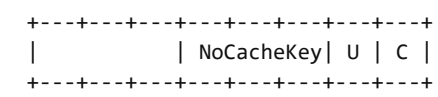



Figure 10: Option Number Mask (Least Significant Byte)

An endpoint may use an equivalent of the C code in Figure 11 to derive the characteristics of an option number "onum".

```
Critical = (onum & 1);
UnSafe = (onum & 2);
NoCacheKey = ((onum & 0x1e) == 0x1c);
```

Figure 11: Determining Characteristics from an Option Number

The option numbers for the options defined in this document are listed in the "CoAP Option Numbers" registry (Section 12.2).

### 5.5. Payloads and Representations

Both requests and responses may include a payload, depending on the Method or Response Code, respectively. If a Method or Response Code is not defined to have a payload, then a sender MUST NOT include one, and a recipient MUST ignore it.

#### 5.5.1. Representation

The payload of requests or of responses indicating success is typically a representation of a resource ("resource representation") or the result of the requested action ("action result"). Its format is specified by the Internet media type and content coding given by the Content-Format Option. In the absence of this option, no default value is assumed, and the format will need to be inferred by the application (e.g., from the application context). Payload "sniffing" SHOULD only be attempted if no content type is given.

Implementation Note: On a quality-of-implementation level, there is a strong expectation that a Content-Format indication will be provided with resource representations whenever possible. This is not a "SHOULD" level requirement solely because it is not a protocol requirement, and it also would be difficult to outline exactly in what cases this expectation can be violated.

For responses indicating a client or server error, the payload is considered a representation of the result of the requested action only if a Content-Format Option is given. In the absence of this option, the payload is a Diagnostic Payload (Section 5.5.2).

### 5.5.2. Diagnostic Payload

If no Content-Format option is given, the payload of responses indicating a client or server error is a brief human-readable diagnostic message, explaining the error situation. This diagnostic message MUST be encoded using UTF-8 [RFC3629], more specifically using Net-Unicode form [RFC5198].

The message is similar to the Reason-Phrase on an HTTP status line. It is not intended for end users but for software engineers that during debugging need to interpret it in the context of the present, English-language specification; therefore, no mechanism for language tagging is needed or provided. In contrast to what is usual in HTTP, the payload SHOULD be empty if there is no additional information beyond the Response Code.

### 5.5.3. Selected Representation

Not all responses carry a payload that provides a representation of the resource addressed by the request. It is, however, sometimes useful to be able to refer to such a representation in relation to a response, independent of whether it actually was enclosed.

We use the term "selected representation" to refer to the current representation of a target resource that would have been selected in a successful response if the corresponding request had used the method GET and excluded any conditional request options (Section 5.10.8).

Certain response options provide metadata about the selected representation, which might differ from the representation included in the message for responses to some state-changing methods. Of the response options defined in this specification, only the ETag response option (Section 5.10.6) is defined as metadata about the selected representation.

### 5.5.4. Content Negotiation

A server may be able to supply a representation for a resource in one of multiple representation formats. Without further information from the client, it will provide the representation in the format it prefers.

By using the Accept Option (Section 5.10.4) in a request, the client can indicate which content-format it prefers to receive.

### 5.6. Caching

CoAP endpoints MAY cache responses in order to reduce the response time and network bandwidth consumption on future, equivalent requests.

The goal of caching in CoAP is to reuse a prior response message to satisfy a current request. In some cases, a stored response can be reused without the need for a network request, reducing latency and network round-trips; a "freshness" mechanism is used for this purpose (see Section 5.6.1). Even when a new request is required, it is often possible to reuse the payload of a prior response to satisfy the request, thereby reducing network bandwidth usage; a "validation" mechanism is used for this purpose (see Section 5.6.2).

Unlike HTTP, the cacheability of CoAP responses does not depend on the request method, but it depends on the Response Code. The cacheability of each Response Code is defined along the Response Code definitions in Section 5.9. Response Codes that indicate success and are unrecognized by an endpoint MUST NOT be cached.

For a presented request, a CoAP endpoint MUST NOT use a stored response, unless:

- the presented request method and that used to obtain the stored response match,
- all options match between those in the presented request and those of the request used to obtain the stored response (which includes the request URI), except that there is no need for a match of any request options marked as NoCacheKey (Section 5.4) or recognized by the Cache and fully interpreted with respect to its specified cache behavior (such as the ETag request option described in Section 5.10.6; see also Section 5.4.2), and
- the stored response is either fresh or successfully validated as defined below.

The set of request options that is used for matching the cache entry is also collectively referred to as the "Cache-Key". For URI schemes other than coap and coaps, matching of those options that constitute the request URI may be performed under rules specific to the URI scheme.

#### 5.6.1. Freshness Model

When a response is "fresh" in the cache, it can be used to satisfy subsequent requests without contacting the origin server, thereby improving efficiency.

The mechanism for determining freshness is for an origin server to provide an explicit expiration time in the future, using the Max-Age Option (see Section 5.10.5). The Max-Age Option indicates that the response is to be considered not fresh after its age is greater than the specified number of seconds.

The Max-Age Option defaults to a value of 60. Thus, if it is not present in a cacheable response, then the response is considered not fresh after its age is greater than 60 seconds. If an origin server wishes to prevent caching, it MUST explicitly include a Max-Age Option with a value of zero seconds.

If a client has a fresh stored response and makes a new request matching the request for that stored response, the new response invalidates the old response.

#### 5.6.2. Validation Model

When an endpoint has one or more stored responses for a GET request, but cannot use any of them (e.g., because they are not fresh), it can use the ETag Option (Section 5.10.6) in the GET request to give the origin server an opportunity both to select a stored response to be used, and to update its freshness. This process is known as "validating" or "revalidating" the stored response.

When sending such a request, the endpoint SHOULD add an ETag Option specifying the entity-tag of each stored response that is applicable.

A 2.03 (Valid) response indicates the stored response identified by the entity-tag given in the response's ETag Option can be reused after updating it as described in Section 5.9.1.3.

Any other Response Code indicates that none of the stored responses nominated in the request is suitable. Instead, the response SHOULD be used to satisfy the request and MAY replace the stored response.

### 5.7. Proxying

A proxy is a CoAP endpoint that can be tasked by CoAP clients to perform requests on their behalf. This may be useful, for example, when the request could otherwise not be made, or to service the response from a cache in order to reduce response time and network bandwidth or energy consumption.

In an overall architecture for a Constrained RESTful Environment, proxies can serve quite different purposes. Proxies can be explicitly selected by clients, a role that we term "forward-proxy". Proxies can also be inserted to stand in for origin servers, a role that we term "reverse-proxy". Orthogonal to this distinction, a proxy can map from a CoAP request to a CoAP request (CoAP-to-CoAP proxy) or translate from or to a different protocol ("cross-proxy"). Full definitions of these terms are provided in Section 1.2.

Notes: The terminology in this specification has been selected to be culturally compatible with the terminology used in the wider web application environments, without necessarily matching it in every detail (which may not even be relevant to Constrained RESTful Environments). Not too much semantics should be ascribed to the components of the terms (such as "forward", "reverse", or "cross").

HTTP proxies, besides acting as HTTP proxies, often offer a transport-protocol proxying function ("CONNECT") to enable end-to-end transport layer security through the proxy. No such function is defined for CoAP-to-CoAP proxies in this specification, as forwarding of UDP packets is unlikely to be of much value in Constrained RESTful Environments. See also Section 10.2.7 for the cross-proxy case.

When a client uses a proxy to make a request that will use a secure URI scheme (e.g., "coaps" or "https"), the request towards the proxy SHOULD be sent using DTLS except where equivalent lower-layer security is used for the leg between the client and the proxy.

#### 5.7.1. Proxy Operation

A proxy generally needs a way to determine potential request parameters for a request it places to a destination, based on the request it received from its client. This way is fully specified for a forward-proxy but may depend on the specific configuration for a reverse-proxy. In particular, the client of a reverse-proxy generally does not indicate a locator for the destination,

necessitating some form of namespace translation in the reverse-proxy. However, some aspects of the operation of proxies are common to all its forms.

If a proxy does not employ a cache, then it simply forwards the translated request to the determined destination. Otherwise, if it does employ a cache but does not have a stored response that matches the translated request and is considered fresh, then it needs to refresh its cache according to Section 5.6. For options in the request that the proxy recognizes, it knows whether the option is intended to act as part of the key used in looking up the cached value or not. For example, since requests for different Uri-Path values address different resources, Uri-Path values are always part of the Cache-Key, while, e.g., Token values are never part of the Cache-Key. For options that the proxy does not recognize but that are marked Safe-to-Forward in the option number, the option also indicates whether it is to be included in the Cache-Key (NoCacheKey is not all set) or not (NoCacheKey is all set). (Options that are unrecognized and marked Unsafe lead to 4.02 Bad Option.)

If the request to the destination times out, then a 5.04 (Gateway Timeout) response MUST be returned. If the request to the destination returns a response that cannot be processed by the proxy (e.g, due to unrecognized critical options or message format errors), then a 5.02 (Bad Gateway) response MUST be returned. Otherwise, the proxy returns the response to the client.

If a response is generated out of a cache, the generated (or implied) Max-Age Option MUST NOT extend the max-age originally set by the server, considering the time the resource representation spent in the cache. For example, the Max-Age Option could be adjusted by the proxy for each response using the formula:

```
proxy-max-age = original-max-age - cache-age
```

For example, if a request is made to a proxied resource that was refreshed 20 seconds ago and had an original Max-Age of 60 seconds, then that resource's proxied max-age is now 40 seconds. Considering potential network delays on the way from the origin server, a proxy should be conservative in the max-age values offered.

All options present in a proxy request MUST be processed at the proxy. Unsafe options in a request that are not recognized by the proxy MUST lead to a 4.02 (Bad Option) response being returned by the proxy. A CoAP-to-CoAP proxy MUST forward to the origin server all Safe-to-Forward options that it does not recognize. Similarly,

Unsafe options in a response that are not recognized by the CoAP-to-CoAP proxy server MUST lead to a 5.02 (Bad Gateway) response. Again, Safe-to-Forward options that are not recognized MUST be forwarded.

Additional considerations for cross-protocol proxying between CoAP and HTTP are discussed in Section 10.

### 5.7.2. Forward-Proxies

CoAP distinguishes between requests made (as if) to an origin server and requests made through a forward-proxy. CoAP requests to a forward-proxy are made as normal Confirmable or Non-confirmable requests to the forward-proxy endpoint, but they specify the request URI in a different way: The request URI in a proxy request is specified as a string in the Proxy-Uri Option (see Section 5.10.2), while the request URI in a request to an origin server is split into the Uri-Host, Uri-Port, Uri-Path, and Uri-Query Options (see Section 5.10.1). Alternatively, the URI in a proxy request can be assembled from a Proxy-Scheme option and the split options mentioned.

When a proxy request is made to an endpoint and the endpoint is unwilling or unable to act as proxy for the request URI, it MUST return a 5.05 (Proxying Not Supported) response. If the authority (host and port) is recognized as identifying the proxy endpoint itself (see Section 5.10.2), then the request MUST be treated as a local (non-proxied) request.

Unless a proxy is configured to forward the proxy request to another proxy, it MUST translate the request as follows: the scheme of the request URI defines the outgoing protocol and its details (e.g., CoAP is used over UDP for the "coap" scheme and over DTLS for the "coaps" scheme.) For a CoAP-to-CoAP proxy, the origin server's IP address and port are determined by the authority component of the request URI, and the request URI is decoded and split into the Uri-Host, Uri-Port, Uri-Path and Uri-Query Options. This consumes the Proxy-Uri or Proxy-Scheme option, which is therefore not forwarded to the origin server.

### 5.7.3. Reverse-Proxies

Reverse-proxies do not make use of the Proxy-Uri or Proxy-Scheme options but need to determine the destination (next hop) of a request from information in the request and information in their configuration. For example, a reverse-proxy might offer various resources as if they were its own resources, after having learned of their existence through resource discovery. The reverse-proxy is free to build a namespace for the URIs that identify these resources. A reverse-proxy may also build a namespace that gives the client more

control over where the request goes, e.g., by embedding host identifiers and port numbers into the URI path of the resources offered.

In processing the response, a reverse-proxy has to be careful that ETag option values from different sources are not mixed up on one resource offered to its clients. In many cases, the ETag can be forwarded unchanged. If the mapping from a resource offered by the reverse-proxy to resources offered by its various origin servers is not unique, the reverse-proxy may need to generate a new ETag, making sure the semantics of this option are properly preserved.

### 5.8. Method Definitions

In this section, each method is defined along with its behavior. A request with an unrecognized or unsupported Method Code MUST generate a 4.05 (Method Not Allowed) piggybacked response.

#### 5.8.1. GET

The GET method retrieves a representation for the information that currently corresponds to the resource identified by the request URI. If the request includes an Accept Option, that indicates the preferred content-format of a response. If the request includes an ETag Option, the GET method requests that ETag be validated and that the representation be transferred only if validation failed. Upon success, a 2.05 (Content) or 2.03 (Valid) Response Code SHOULD be present in the response.

The GET method is safe and idempotent.

#### 5.8.2. POST

The POST method requests that the representation enclosed in the request be processed. The actual function performed by the POST method is determined by the origin server and dependent on the target resource. It usually results in a new resource being created or the target resource being updated.

If a resource has been created on the server, the response returned by the server SHOULD have a 2.01 (Created) Response Code and SHOULD include the URI of the new resource in a sequence of one or more Location-Path and/or Location-Query Options (Section 5.10.7). If the POST succeeds but does not result in a new resource being created on the server, the response SHOULD have a 2.04 (Changed) Response Code. If the POST succeeds and results in the target resource being deleted, the response SHOULD have a 2.02 (Deleted) Response Code. POST is neither safe nor idempotent.

#### 5.8.3. PUT

The PUT method requests that the resource identified by the request URI be updated or created with the enclosed representation. The representation format is specified by the media type and content coding given in the Content-Format Option, if provided.

If a resource exists at the request URI, the enclosed representation SHOULD be considered a modified version of that resource, and a 2.04 (Changed) Response Code SHOULD be returned. If no resource exists, then the server MAY create a new resource with that URI, resulting in a 2.01 (Created) Response Code. If the resource could not be created or modified, then an appropriate error Response Code SHOULD be sent.

Further restrictions to a PUT can be made by including the If-Match (see Section 5.10.8.1) or If-None-Match (see Section 5.10.8.2) options in the request.

PUT is not safe but is idempotent.

#### 5.8.4. DELETE

The DELETE method requests that the resource identified by the request URI be deleted. A 2.02 (Deleted) Response Code SHOULD be used on success or in case the resource did not exist before the request.

DELETE is not safe but is idempotent.

### 5.9. Response Code Definitions

Each Response Code is described below, including any options required in the response. Where appropriate, some of the codes will be specified in regards to related Response Codes in HTTP [RFC2616]; this does not mean that any such relationship modifies the HTTP mapping specified in Section 10.

#### 5.9.1. Success 2.xx

This class of Response Code indicates that the clients request was successfully received, understood, and accepted.

##### 5.9.1.1. 2.01 Created

Like HTTP 201 "Created", but only used in response to POST and PUT requests. The payload returned with the response, if any, is a representation of the action result.

If the response includes one or more Location-Path and/or Location-Query Options, the values of these options specify the location at which the resource was created. Otherwise, the resource was created at the request URI. A cache receiving this response MUST mark any stored response for the created resource as not fresh.

This response is not cacheable.

#### 5.9.1.2. 2.02 Deleted

This Response Code is like HTTP 204 "No Content" but only used in response to requests that cause the resource to cease being available, such as DELETE and, in certain circumstances, POST. The payload returned with the response, if any, is a representation of the action result.

This response is not cacheable. However, a cache MUST mark any stored response for the deleted resource as not fresh.

#### 5.9.1.3. 2.03 Valid

This Response Code is related to HTTP 304 "Not Modified" but only used to indicate that the response identified by the entity-tag identified by the included ETag Option is valid. Accordingly, the response MUST include an ETag Option and MUST NOT include a payload.

When a cache that recognizes and processes the ETag response option receives a 2.03 (Valid) response, it MUST update the stored response with the value of the Max-Age Option included in the response (explicitly, or implicitly as a default value; see also Section 5.6.2). For each type of Safe-to-Forward option present in the response, the (possibly empty) set of options of this type that are present in the stored response MUST be replaced with the set of options of this type in the response received. (Unsafe options may trigger similar option-specific processing as defined by the option.)

#### 5.9.1.4. 2.04 Changed

This Response Code is like HTTP 204 "No Content" but only used in response to POST and PUT requests. The payload returned with the response, if any, is a representation of the action result.

This response is not cacheable. However, a cache MUST mark any stored response for the changed resource as not fresh.

#### 5.9.1.5. 2.05 Content

This Response Code is like HTTP 200 "OK" but only used in response to GET requests.

The payload returned with the response is a representation of the target resource.

This response is cacheable: Caches can use the Max-Age Option to determine freshness (see Section 5.6.1) and (if present) the ETag Option for validation (see Section 5.6.2).

### 5.9.2. Client Error 4.xx

This class of Response Code is intended for cases in which the client seems to have erred. These Response Codes are applicable to any request method.

The server SHOULD include a diagnostic payload under the conditions detailed in Section 5.5.2.

Responses of this class are cacheable: Caches can use the Max-Age Option to determine freshness (see Section 5.6.1). They cannot be validated.

#### 5.9.2.1. 4.00 Bad Request

This Response Code is Like HTTP 400 "Bad Request".

#### 5.9.2.2. 4.01 Unauthorized

The client is not authorized to perform the requested action. The client SHOULD NOT repeat the request without first improving its authentication status to the server. Which specific mechanism can be used for this is outside this document's scope; see also Section 9.

#### 5.9.2.3. 4.02 Bad Option

The request could not be understood by the server due to one or more unrecognized or malformed options. The client SHOULD NOT repeat the request without modification.

#### 5.9.2.4. 4.03 Forbidden

This Response Code is like HTTP 403 "Forbidden".

#### 5.9.2.5. 4.04 Not Found

This Response Code is like HTTP 404 "Not Found".

#### 5.9.2.6. 4.05 Method Not Allowed

This Response Code is like HTTP 405 "Method Not Allowed" but with no parallel to the "Allow" header field.

#### 5.9.2.7. 4.06 Not Acceptable

This Response Code is like HTTP 406 "Not Acceptable", but with no response entity.

#### 5.9.2.8. 4.12 Precondition Failed

This Response Code is like HTTP 412 "Precondition Failed".

#### 5.9.2.9. 4.13 Request Entity Too Large

This Response Code is like HTTP 413 "Request Entity Too Large".

The response SHOULD include a Size1 Option (Section 5.10.9) to indicate the maximum size of request entity the server is able and willing to handle, unless the server is not in a position to make this information available.

#### 5.9.2.10. 4.15 Unsupported Content-Format

This Response Code is like HTTP 415 "Unsupported Media Type".

### 5.9.3. Server Error 5.xx

This class of Response Code indicates cases in which the server is aware that it has erred or is incapable of performing the request. These Response Codes are applicable to any request method.

The server SHOULD include a diagnostic payload under the conditions detailed in Section 5.5.2.

Responses of this class are cacheable: Caches can use the Max-Age Option to determine freshness (see Section 5.6.1). They cannot be validated.

#### 5.9.3.1. 5.00 Internal Server Error

This Response Code is like HTTP 500 "Internal Server Error".

#### 5.9.3.2. 5.01 Not Implemented

This Response Code is like HTTP 501 "Not Implemented".

#### 5.9.3.3. 5.02 Bad Gateway

This Response Code is like HTTP 502 "Bad Gateway".

#### 5.9.3.4. 5.03 Service Unavailable

This Response Code is like HTTP 503 "Service Unavailable" but uses the Max-Age Option in place of the "Retry-After" header field to indicate the number of seconds after which to retry.

#### 5.9.3.5. 5.04 Gateway Timeout

This Response Code is like HTTP 504 "Gateway Timeout".

#### 5.9.3.6. 5.05 Proxying Not Supported

The server is unable or unwilling to act as a forward-proxy for the URI specified in the Proxy-Uri Option or using Proxy-Scheme (see Section 5.10.2).

### 5.10. Option Definitions

The individual CoAP options are summarized in Table 4 and explained in the subsections of this section.

In this table, the C, U, and N columns indicate the properties Critical, UnSafe, and NoCacheKey, respectively. Since NoCacheKey only has a meaning for options that are Safe-to-Forward (not marked Unsafe), the column is filled with a dash for UnSafe options.

| No. | C | U | N | R | Name | Format | Length | Default |
|---|---|---|---|---|---|---|---|---|
| 1 | x | | | x | If-Match | opaque | 0-8 | (none) |
| 3 | x | x | - | | Uri-Host | string | 1-255 | (see below) |
| 4 | | | | x | ETag | opaque | 1-8 | (none) |
| 5 | x | | | | If-None-Match | empty | 0 | (none) |
| 7 | x | x | - | | Uri-Port | uint | 0-2 | (see below) |
| 8 | | | | x | Location-Path | string | 0-255 | (none) |
| 11 | x | x | - | x | Uri-Path | string | 0-255 | (none) |
| 12 | | | | | Content-Format | uint | 0-2 | (none) |
| 14 | | x | - | | Max-Age | uint | 0-4 | 60 |
| 15 | x | x | - | x | Uri-Query | string | 0-255 | (none) |
| 17 | x | | | | Accept | uint | 0-2 | (none) |
| 20 | | | | x | Location-Query | string | 0-255 | (none) |
| 35 | x | x | - | | Proxy-Uri | string | 1-1034 | (none) |
| 39 | x | x | - | | Proxy-Scheme | string | 1-255 | (none) |
| 60 | | | x | | Size1 | uint | 0-4 | (none) |

C=Critical, U=Unsafe, N=NoCacheKey, R=Repeatable

Table 4: Options

### 5.10.1. Uri-Host, Uri-Port, Uri-Path, and Uri-Query

The Uri-Host, Uri-Port, Uri-Path, and Uri-Query Options are used to specify the target resource of a request to a CoAP origin server. The options encode the different components of the request URI in a way that no percent-encoding is visible in the option values and that the full URI can be reconstructed at any involved endpoint. The syntax of CoAP URIs is defined in Section 6.

The steps for parsing URIs into options is defined in Section 6.4. These steps result in zero or more Uri-Host, Uri-Port, Uri-Path, and Uri-Query Options being included in a request, where each option holds the following values:

- the Uri-Host Option specifies the Internet host of the resource being requested,
- the Uri-Port Option specifies the transport-layer port number of the resource,
- each Uri-Path Option specifies one segment of the absolute path to the resource, and

- each Uri-Query Option specifies one argument parameterizing the resource.

Note: Fragments ([RFC3986], Section 3.5) are not part of the request URI and thus will not be transmitted in a CoAP request.

The default value of the Uri-Host Option is the IP literal representing the destination IP address of the request message. Likewise, the default value of the Uri-Port Option is the destination UDP port. The default values for the Uri-Host and Uri-Port Options are sufficient for requests to most servers. Explicit Uri-Host and Uri-Port Options are typically used when an endpoint hosts multiple virtual servers.

The Uri-Path and Uri-Query Option can contain any character sequence. No percent-encoding is performed. The value of a Uri-Path Option MUST NOT be "." or ".." (as the request URI must be resolved before parsing it into options).

The steps for constructing the request URI from the options are defined in Section 6.5. Note that an implementation does not necessarily have to construct the URI; it can simply look up the target resource by examining the individual options.

Examples can be found in Appendix B.

#### 5.10.2. Proxy-Uri and Proxy-Scheme

The Proxy-Uri Option is used to make a request to a forward-proxy (see Section 5.7). The forward-proxy is requested to forward the request or service it from a valid cache and return the response.

The option value is an absolute-URI ([RFC3986], Section 4.3).

Note that the forward-proxy MAY forward the request on to another proxy or directly to the server specified by the absolute-URI. In order to avoid request loops, a proxy MUST be able to recognize all of its server names, including any aliases, local variations, and the numeric IP addresses.

An endpoint receiving a request with a Proxy-Uri Option that is unable or unwilling to act as a forward-proxy for the request MUST cause the return of a 5.05 (Proxying Not Supported) response.

The Proxy-Uri Option MUST take precedence over any of the Uri-Host, Uri-Port, Uri-Path or Uri-Query options (each of which MUST NOT be included in a request containing the Proxy-Uri Option).

As a special case to simplify many proxy clients, the absolute-URI can be constructed from the Uri-* options. When a Proxy-Scheme Option is present, the absolute-URI is constructed as follows: a CoAP URI is constructed from the Uri-* options as defined in Section 6.5. In the resulting URI, the initial scheme up to, but not including, the following colon is then replaced by the content of the Proxy-Scheme Option. Note that this case is only applicable if the components of the desired URI other than the scheme component actually can be expressed using Uri-* options; for example, to represent a URI with a userinfo component in the authority, only Proxy-Uri can be used.

#### 5.10.3. Content-Format

The Content-Format Option indicates the representation format of the message payload. The representation format is given as a numeric Content-Format identifier that is defined in the "CoAP Content-Formats" registry (Section 12.3). In the absence of the option, no default value is assumed, i.e., the representation format of any representation message payload is indeterminate (Section 5.5).

#### 5.10.4. Accept

The CoAP Accept option can be used to indicate which Content-Format is acceptable to the client. The representation format is given as a numeric Content-Format identifier that is defined in the "CoAP Content-Formats" registry (Section 12.3). If no Accept option is given, the client does not express a preference (thus no default value is assumed). The client prefers the representation returned by the server to be in the Content-Format indicated. The server returns the preferred Content-Format if available. If the preferred Content-Format cannot be returned, then a 4.06 "Not Acceptable" MUST be sent as a response, unless another error code takes precedence for this response.

#### 5.10.5. Max-Age

The Max-Age Option indicates the maximum time a response may be cached before it is considered not fresh (see Section 5.6.1).

The option value is an integer number of seconds between 0 and 2**32-1 inclusive (about 136.1 years). A default value of 60 seconds is assumed in the absence of the option in a response.

The value is intended to be current at the time of transmission. Servers that provide resources with strict tolerances on the value of Max-Age SHOULD update the value before each retransmission. (See also Section 5.7.1.)

#### 5.10.6. ETag

An entity-tag is intended for use as a resource-local identifier for differentiating between representations of the same resource that vary over time. It is generated by the server providing the resource, which may generate it in any number of ways including a version, checksum, hash, or time. An endpoint receiving an entity-tag MUST treat it as opaque and make no assumptions about its content or structure. (Endpoints that generate an entity-tag are encouraged to use the most compact representation possible, in particular in regards to clients and intermediaries that may want to store multiple ETag values.)

##### 5.10.6.1. ETag as a Response Option

The ETag Option in a response provides the current value (i.e., after the request was processed) of the entity-tag for the "tagged representation". If no Location-* options are present, the tagged representation is the selected representation (Section 5.5.3) of the target resource. If one or more Location-* options are present and thus a location URI is indicated (Section 5.10.7), the tagged representation is the representation that would be retrieved by a GET request to the location URI.

An ETag response option can be included with any response for which there is a tagged representation (e.g., it would not be meaningful in a 4.04 or 4.00 response). The ETag Option MUST NOT occur more than once in a response.

There is no default value for the ETag Option; if it is not present in a response, the server makes no statement about the entity-tag for the tagged representation.

##### 5.10.6.2. ETag as a Request Option

In a GET request, an endpoint that has one or more representations previously obtained from the resource, and has obtained ETag response options with these, can specify an instance of the ETag Option for one or more of these stored responses.

A server can issue a 2.03 Valid response (Section 5.9.1.3) in place of a 2.05 Content response if one of the ETags given is the entity-tag for the current representation, i.e., is valid; the 2.03 Valid response then echoes this specific ETag in a response option.

In effect, a client can determine if any of the stored representations is current (see Section 5.6.2) without needing to transfer them again.

The ETag Option MAY occur zero, one, or multiple times in a request.

### 5.10.7. Location-Path and Location-Query

The Location-Path and Location-Query Options together indicate a relative URI that consists either of an absolute path, a query string, or both. A combination of these options is included in a 2.01 (Created) response to indicate the location of the resource created as the result of a POST request (see Section 5.8.2). The location is resolved relative to the request URI.

If a response with one or more Location-Path and/or Location-Query Options passes through a cache that interprets these options and the implied URI identifies one or more currently stored responses, those entries MUST be marked as not fresh.

Each Location-Path Option specifies one segment of the absolute path to the resource, and each Location-Query Option specifies one argument parameterizing the resource. The Location-Path and Location-Query Option can contain any character sequence. No percent-encoding is performed. The value of a Location-Path Option MUST NOT be "." or "..".

The steps for constructing the location URI from the options are analogous to Section 6.5, except that the first five steps are skipped and the result is a relative URI-reference, which is then interpreted relative to the request URI. Note that the relative URI-reference constructed this way always includes an absolute path (e.g., leaving out Location-Path but supplying Location-Query means the path component in the URI is "/").

The options that are used to compute the relative URI-reference are collectively called Location-* options. Beyond Location-Path and Location-Query, more Location-* options may be defined in the future and have been reserved option numbers 128, 132, 136, and 140. If any of these reserved option numbers occurs in addition to Location-Path and/or Location-Query and are not supported, then a 4.02 (Bad Option) error MUST be returned.

### 5.10.8. Conditional Request Options

Conditional request options enable a client to ask the server to perform the request only if certain conditions specified by the option are fulfilled.

For each of these options, if the condition given is not fulfilled, then the server MUST NOT perform the requested method. Instead, the server MUST respond with the 4.12 (Precondition Failed) Response Code.

If the condition is fulfilled, the server performs the request method as if the conditional request options were not present.

If the request would, without the conditional request options, result in anything other than a 2.xx or 4.12 Response Code, then any conditional request options MAY be ignored.

#### 5.10.8.1. If-Match

The If-Match Option MAY be used to make a request conditional on the current existence or value of an ETag for one or more representations of the target resource. If-Match is generally useful for resource update requests, such as PUT requests, as a means for protecting against accidental overwrites when multiple clients are acting in parallel on the same resource (i.e., the "lost update" problem).

The value of an If-Match option is either an ETag or the empty string. An If-Match option with an ETag matches a representation with that exact ETag. An If-Match option with an empty value matches any existing representation (i.e., it places the precondition on the existence of any current representation for the target resource).

The If-Match Option can occur multiple times. If any of the options match, then the condition is fulfilled.

If there is one or more If-Match Options, but none of the options match, then the condition is not fulfilled.

#### 5.10.8.2. If-None-Match

The If-None-Match Option MAY be used to make a request conditional on the nonexistence of the target resource. If-None-Match is useful for resource creation requests, such as PUT requests, as a means for protecting against accidental overwrites when multiple clients are acting in parallel on the same resource. The If-None-Match Option carries no value.

If the target resource does exist, then the condition is not fulfilled.

(It is not very useful to combine If-Match and If-None-Match options in one request, because the condition will then never be fulfilled.)

### 5.10.9. Size1 Option

The Size1 option provides size information about the resource representation in a request. The option value is an integer number of bytes. Its main use is with block-wise transfers [BLOCK]. In the present specification, it is used in 4.13 responses (Section 5.9.2.9) to indicate the maximum size of request entity that the server is able and willing to handle.

## 6. CoAP URIs

CoAP uses the "coap" and "coaps" URI schemes for identifying CoAP resources and providing a means of locating the resource. Resources are organized hierarchically and governed by a potential CoAP origin server listening for CoAP requests ("coap") or DTLS-secured CoAP requests ("coaps") on a given UDP port. The CoAP server is identified via the generic syntax's authority component, which includes a host component and optional UDP port number. The remainder of the URI is considered to be identifying a resource that can be operated on by the methods defined by the CoAP protocol. The "coap" and "coaps" URI schemes can thus be compared to the "http" and "https" URI schemes, respectively.

The syntax of the "coap" and "coaps" URI schemes is specified in this section in Augmented Backus-Naur Form (ABNF) [RFC5234]. The definitions of "host", "port", "path-abempty", "query", "segment", "IP-literal", "IPv4address", and "reg-name" are adopted from [RFC3986].

Implementation Note: Unfortunately, over time, the URI format has acquired significant complexity. Implementers are encouraged to examine [RFC3986] closely. For example, the ABNF for IPv6 addresses is more complicated than maybe expected. Also, implementers should take care to perform the processing of percent-decoding or percent-encoding exactly once on the way from a URI to its decoded components or back. Percent-encoding is crucial for data transparency but may lead to unusual results such as a slash character in a path component.

### 6.1. coap URI Scheme

```
coap-URI = "coap:" "//" host [ ":" port ] path-abempty [ "?" query ]
```

If the host component is provided as an IP-literal or IPv4address, then the CoAP server can be reached at that IP address. If host is a registered name, then that name is considered an indirect identifier and the endpoint might use a name resolution service, such as DNS, to find the address of that host. The host MUST NOT be empty; if a URI

is received with a missing authority or an empty host, then it MUST be considered invalid. The port subcomponent indicates the UDP port at which the CoAP server is located. If it is empty or not given, then the default port 5683 is assumed.

The path identifies a resource within the scope of the host and port. It consists of a sequence of path segments separated by a slash character (U+002F SOLIDUS "/").

The query serves to further parameterize the resource. It consists of a sequence of arguments separated by an ampersand character (U+0026 AMPERSAND "&"). An argument is often in the form of a "key=value" pair.

The "coap" URI scheme supports the path prefix "/.well-known/" defined by [RFC5785] for "well-known locations" in the namespace of a host. This enables discovery of policy or other information about a host ("site-wide metadata"), such as hosted resources (see Section 7).

Application designers are encouraged to make use of short but descriptive URIs. As the environments that CoAP is used in are usually constrained for bandwidth and energy, the trade-off between these two qualities should lean towards the shortness, without ignoring descriptiveness.

### 6.2. coaps URI Scheme

```
coaps-URI = "coaps:" "//" host [ ":" port ] path-abempty
            [ "?" query ]
```

All of the requirements listed above for the "coap" scheme are also requirements for the "coaps" scheme, except that a default UDP port of 5684 is assumed if the port subcomponent is empty or not given, and the UDP datagrams MUST be secured through the use of DTLS as described in Section 9.1.

Considerations for caching of responses to "coaps" identified requests are discussed in Section 11.2.

Resources made available via the "coaps" scheme have no shared identity with the "coap" scheme even if their resource identifiers indicate the same authority (the same host listening to the same UDP port). They are distinct namespaces and are considered to be distinct origin servers.

### 6.3. Normalization and Comparison Rules

Since the "coap" and "coaps" schemes conform to the URI generic syntax, such URIs are normalized and compared according to the algorithm defined in [RFC3986], Section 6, using the defaults described above for each scheme.

If the port is equal to the default port for a scheme, the normal form is to elide the port subcomponent. Likewise, an empty path component is equivalent to an absolute path of "/", so the normal form is to provide a path of "/" instead. The scheme and host are case insensitive and normally provided in lowercase; IP-literals are in recommended form [RFC5952]; all other components are compared in a case-sensitive manner. Characters other than those in the "reserved" set are equivalent to their percent-encoded bytes (see [RFC3986], Section 2.1): the normal form is to not encode them.

For example, the following three URIs are equivalent and cause the same options and option values to appear in the CoAP messages:

coap://example.com:5683/~sensors/temp.xml
coap://EXAMPLE.com/%7Esensors/temp.xml
coap://EXAMPLE.com:/%7esensors/temp.xml

### 6.4. Decomposing URIs into Options

The steps to parse a request's options from a string |url| are as follows. These steps either result in zero or more of the Uri-Host, Uri-Port, Uri-Path, and Uri-Query Options being included in the request or they fail.

1. If the |url| string is not an absolute URI ([RFC3986]), then fail this algorithm.

2. Resolve the |url| string using the process of reference resolution defined by [RFC3986]. At this stage, the URL is in ASCII encoding [RFC0020], even though the decoded components will be interpreted in UTF-8 [RFC3629] after steps 5, 8, and 9.

   NOTE: It doesn't matter what it is resolved relative to, since we already know it is an absolute URL at this point.

3. If |url| does not have a <scheme> component whose value, when converted to ASCII lowercase, is "coap" or "coaps", then fail this algorithm.

4. If |url| has a <fragment> component, then fail this algorithm.

5. If the <host> component of |url| does not represent the request's destination IP address as an IP-literal or IPv4address, include a Uri-Host Option and let that option's value be the value of the <host> component of |url|, converted to ASCII lowercase, and then convert all percent-encodings ("%" followed by two hexadecimal digits) to the corresponding characters.

   NOTE: In the usual case where the request's destination IP address is derived from the host part, this ensures that a Uri-Host Option is only used for a <host> component of the form reg-name.

6. If |url| has a <port> component, then let |port| be that component's value interpreted as a decimal integer; otherwise, let |port| be the default port for the scheme.

7. If |port| does not equal the request's destination UDP port, include a Uri-Port Option and let that option's value be |port|.

8. If the value of the <path> component of |url| is empty or consists of a single slash character (U+002F SOLIDUS "/"), then move to the next step.

   Otherwise, for each segment in the <path> component, include a Uri-Path Option and let that option's value be the segment (not including the delimiting slash characters) after converting each percent-encoding ("%" followed by two hexadecimal digits) to the corresponding byte.

9. If |url| has a <query> component, then, for each argument in the <query> component, include a Uri-Query Option and let that option's value be the argument (not including the question mark and the delimiting ampersand characters) after converting each percent-encoding to the corresponding byte.

Note that these rules completely resolve any percent-encoding.

### 6.5. Composing URIs from Options

The steps to construct a URI from a request's options are as follows. These steps either result in a URI or they fail. In these steps, percent-encoding a character means replacing each of its (UTF-8-encoded) bytes by a "%" character followed by two hexadecimal digits representing the byte, where the digits A-F are in uppercase (as defined in Section 2.1 of [RFC3986]; to reduce variability, the hexadecimal notation for percent-encoding in CoAP URIs MUST use uppercase letters). The definitions of "unreserved" and "sub-delims" are adopted from [RFC3986].

1. If the request is secured using DTLS, let |url| be the string "coaps://". Otherwise, let |url| be the string "coap://".

2. If the request includes a Uri-Host Option, let |host| be that option's value, where any non-ASCII characters are replaced by their corresponding percent-encoding. If |host| is not a valid reg-name or IP-literal or IPv4address, fail the algorithm. If the request does not include a Uri-Host Option, let |host| be the IP-literal (making use of the conventions of [RFC5952]) or IPv4address representing the request's destination IP address.

3. Append |host| to |url|.

4. If the request includes a Uri-Port Option, let |port| be that option's value. Otherwise, let |port| be the request's destination UDP port.

5. If |port| is not the default port for the scheme, then append a single U+003A COLON character (:) followed by the decimal representation of |port| to |url|.

6. Let |resource name| be the empty string. For each Uri-Path Option in the request, append a single character U+002F SOLIDUS (/) followed by the option's value to |resource name|, after converting any character that is not either in the "unreserved" set, in the "sub-delims" set, a U+003A COLON (:) character, or a U+0040 COMMERCIAL AT (@) character to its percent-encoded form.

7. If |resource name| is the empty string, set it to a single character U+002F SOLIDUS (/).

8. For each Uri-Query Option in the request, append a single character U+003F QUESTION MARK (?) (first option) or U+0026 AMPERSAND (&) (subsequent options) followed by the option's value to |resource name|, after converting any character that is not either in the "unreserved" set, in the "sub-delims" set (except U+0026 AMPERSAND (&)), a U+003A COLON (:), a U+0040 COMMERCIAL AT (@), a U+002F SOLIDUS (/), or a U+003F QUESTION MARK (?) character to its percent-encoded form.

9. Append |resource name| to |url|.

10. Return |url|.

Note that these steps have been designed to lead to a URI in normal form (see Section 6.3).

# 7. Discovery

## 7.1. Service Discovery

As a part of discovering the services offered by a CoAP server, a client has to learn about the endpoint used by a server.

A server is discovered by a client (knowing or) learning a URI that references a resource in the namespace of the server. Alternatively, clients can use multicast CoAP (see Section 8) and the "All CoAP Nodes" multicast address to find CoAP servers.

Unless the port subcomponent in a "coap" or "coaps" URI indicates the UDP port at which the CoAP server is located, the server is assumed to be reachable at the default port.

The CoAP default port number 5683 MUST be supported by a server that offers resources for resource discovery (see Section 7.2 below) and SHOULD be supported for providing access to other resources. The default port number 5684 for DTLS-secured CoAP MAY be supported by a server for resource discovery and for providing access to other resources. In addition, other endpoints may be hosted at other ports, e.g., in the dynamic port space.

Implementation Note: When a CoAP server is hosted by a 6LoWPAN node, header compression efficiency is improved when it also supports a port number in the 61616-61631 compressed UDP port space defined in [RFC4944] and [RFC6282]. (Note that, as its UDP port differs from the default port, it is a different endpoint from the server at the default port.)

## 7.2. Resource Discovery

The discovery of resources offered by a CoAP endpoint is extremely important in machine-to-machine applications where there are no humans in the loop and static interfaces result in fragility. To maximize interoperability in a CoRE environment, a CoAP endpoint SHOULD support the CoRE Link Format of discoverable resources as described in [RFC6690], except where fully manual configuration is desired. It is up to the server which resources are made discoverable (if any).

### 7.2.1. 'ct' Attribute

This section defines a new Web Linking [RFC5988] attribute for use with [RFC6690]. The Content-Format code "ct" attribute provides a hint about the Content-Formats this resource returns. Note that this is only a hint, and it does not override the Content-Format Option of

a CoAP response obtained by actually requesting the representation of the resource. The value is in the CoAP identifier code format as a decimal ASCII integer and MUST be in the range of 0-65535 (16-bit unsigned integer). For example, "application/xml" would be indicated as "ct=41". If no Content-Format code attribute is present, then nothing about the type can be assumed. The Content-Format code attribute MAY include a space-separated sequence of Content-Format codes, indicating that multiple content-formats are available. The syntax of the attribute value is summarized in the production "ct-value" in Figure 12, where "cardinal", "SP", and "DQUOTE" are defined as in [RFC6690].

```
ct-value =  cardinal
         /  DQUOTE cardinal *( 1*SP cardinal ) DQUOTE
```

Figure 12

# 8. Multicast CoAP

CoAP supports making requests to an IP multicast group. This is defined by a series of deltas to unicast CoAP. A more general discussion of group communication with CoAP is in [GROUPCOMM].

CoAP endpoints that offer services that they want other endpoints to be able to find using multicast service discovery join one or more of the appropriate all-CoAP-node multicast addresses (Section 12.8) and listen on the default CoAP port. Note that an endpoint might receive multicast requests on other multicast addresses, including the all-nodes IPv6 address (or via broadcast on IPv4); an endpoint MUST therefore be prepared to receive such messages but MAY ignore them if multicast service discovery is not desired.

## 8.1. Messaging Layer

A multicast request is characterized by being transported in a CoAP message that is addressed to an IP multicast address instead of a CoAP endpoint. Such multicast requests MUST be Non-confirmable.

A server SHOULD be aware that a request arrived via multicast, e.g., by making use of modern APIs such as IPV6_RECVPKTINFO [RFC3542], if available.

To avoid an implosion of error responses, when a server is aware that a request arrived via multicast, it MUST NOT return a Reset message in reply to a Non-confirmable message. If it is not aware, it MAY return a Reset message in reply to a Non-confirmable message as usual. Because such a Reset message will look identical to one for a

unicast message from the sender, the sender MUST avoid using a Message ID that is also still active from this endpoint with any unicast endpoint that might receive the multicast message.

At the time of writing, multicast messages can only be carried in UDP not in DTLS. This means that the security modes defined for CoAP in this document are not applicable to multicast.

### 8.2. Request/Response Layer

When a server is aware that a request arrived via multicast, the server MAY always ignore the request, in particular if it doesn't have anything useful to respond (e.g., if it only has an empty payload or an error response). The decision for this may depend on the application. (For example, in query filtering as described in [RFC6690], a server should not respond to a multicast request if the filter does not match. More examples are in [GROUPCOMM].)

If a server does decide to respond to a multicast request, it should not respond immediately. Instead, it should pick a duration for the period of time during which it intends to respond. For the purposes of this exposition, we call the length of this period the Leisure. The specific value of this Leisure may depend on the application or MAY be derived as described below. The server SHOULD then pick a random point of time within the chosen leisure period to send back the unicast response to the multicast request. If further responses need to be sent based on the same multicast address membership, a new leisure period starts at the earliest after the previous one finishes.

To compute a value for Leisure, the server should have a group size estimate G, a target data transfer rate R (which both should be chosen conservatively), and an estimated response size S; a rough lower bound for Leisure can then be computed as

$$lb_Leisure = S * G / R$$

For example, for a multicast request with link-local scope on a 2.4 GHz IEEE 802.15.4 (6LoWPAN) network, G could be (relatively conservatively) set to 100, S to 100 bytes, and the target rate to 8 kbit/s = 1 kB/s. The resulting lower bound for the Leisure is 10 seconds.

If a CoAP endpoint does not have suitable data to compute a value for Leisure, it MAY resort to DEFAULT_LEISURE.

When matching a response to a multicast request, only the token MUST match; the source endpoint of the response does not need to (and will not) be the same as the destination endpoint of the original request.

For the purposes of interpreting the Location-* options and any links embedded in the representation, the request URI (i.e., the base URI relative to which the response is interpreted) is formed by replacing the multicast address in the Host component of the original request URI by the literal IP address of the endpoint actually responding.

#### 8.2.1. Caching

When a client makes a multicast request, it always makes a new request to the multicast group (since there may be new group members that joined meanwhile or ones that did not get the previous request). It MAY update a cache with the received responses. Then, it uses both cached-still-fresh and new responses as the result of the request.

A response received in reply to a GET request to a multicast group MAY be used to satisfy a subsequent request on the related unicast request URI. The unicast request URI is obtained by replacing the authority part of the request URI with the transport-layer source address of the response message.

A cache MAY revalidate a response by making a GET request on the related unicast request URI.

A GET request to a multicast group MUST NOT contain an ETag option. A mechanism to suppress responses the client already has is left for further study.

#### 8.2.2. Proxying

When a forward-proxy receives a request with a Proxy-Uri or URI constructed from Proxy-Scheme that indicates a multicast address, the proxy obtains a set of responses as described above and sends all responses (both cached-still-fresh and new) back to the original client.

This specification does not provide a way to indicate the unicast-modified request URI (base URI) in responses thus forwarded. Proxying multicast requests is discussed in more detail in [GROUPCOMM]; one proposal to address the base URI issue can be found in Section 3 of [CoAP-MISC].

# 9. Securing CoAP

This section defines the DTLS binding for CoAP.

During the provisioning phase, a CoAP device is provided with the security information that it needs, including keying materials and access control lists. This specification defines provisioning for the RawPublicKey mode in Section 9.1.3.2.1. At the end of the provisioning phase, the device will be in one of four security modes with the following information for the given mode. The NoSec and RawPublicKey modes are mandatory to implement for this specification.

NoSec: There is no protocol-level security (DTLS is disabled). Alternative techniques to provide lower-layer security SHOULD be used when appropriate. The use of IPsec is discussed in [IPsec-CoAP]. Certain link layers in use with constrained nodes also provide link-layer security, which may be appropriate with proper key management.

PreSharedKey: DTLS is enabled, there is a list of pre-shared keys [RFC4279], and each key includes a list of which nodes it can be used to communicate with as described in Section 9.1.3.1. At the extreme, there may be one key for each node this CoAP node needs to communicate with (1:1 node/key ratio). Conversely, if more than two entities share a specific pre-shared key, this key only enables the entities to authenticate as a member of that group and not as a specific peer.

RawPublicKey: DTLS is enabled and the device has an asymmetric key pair without a certificate (a raw public key) that is validated using an out-of-band mechanism [RFC7250] as described in Section 9.1.3.2. The device also has an identity calculated from the public key and a list of identities of the nodes it can communicate with.

Certificate: DTLS is enabled and the device has an asymmetric key pair with an X.509 certificate [RFC5280] that binds it to its subject and is signed by some common trust root as described in Section 9.1.3.3. The device also has a list of root trust anchors that can be used for validating a certificate.

In the "NoSec" mode, the system simply sends the packets over normal UDP over IP and is indicated by the "coap" scheme and the CoAP default port. The system is secured only by keeping attackers from being able to send or receive packets from the network with the CoAP nodes; see Section 11.5 for an additional complication with this approach.

The other three security modes are achieved using DTLS and are indicated by the "coaps" scheme and DTLS-secured CoAP default port. The result is a security association that can be used to authenticate (within the limits of the security model) and, based on this authentication, authorize the communication partner. CoAP itself does not provide protocol primitives for authentication or authorization; where this is required, it can either be provided by communication security (i.e., IPsec or DTLS) or by object security (within the payload). Devices that require authorization for certain operations are expected to require one of these two forms of security. Necessarily, where an intermediary is involved, communication security only works when that intermediary is part of the trust relationships. CoAP does not provide a way to forward different levels of authorization that clients may have with an intermediary to further intermediaries or origin servers -- it therefore may be required to perform all authorization at the first intermediary.

## 9.1. DTLS-Secured CoAP

Just as HTTP is secured using Transport Layer Security (TLS) over TCP, CoAP is secured using Datagram TLS (DTLS) [RFC6347] over UDP (see Figure 13). This section defines the CoAP binding to DTLS, along with the minimal mandatory-to-implement configurations appropriate for constrained environments. The binding is defined by a series of deltas to unicast CoAP. In practice, DTLS is TLS with added features to deal with the unreliable nature of the UDP transport.

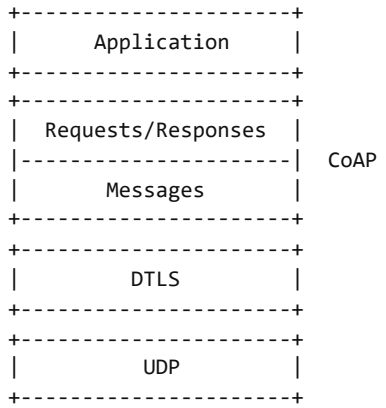



Figure 13: Abstract Layering of DTLS-Secured CoAP

In some constrained nodes (limited flash and/or RAM) and networks (limited bandwidth or high scalability requirements), and depending on the specific cipher suites in use, all modes of DTLS may not be applicable. Some DTLS cipher suites can add significant implementation complexity as well as some initial handshake overhead needed when setting up the security association. Once the initial handshake is completed, DTLS adds a limited per-datagram overhead of approximately 13 bytes, not including any initialization vectors/ nonces (e.g., 8 bytes with TLS_PSK_WITH_AES_128_CCM_8 [RFC6655]), integrity check values (e.g., 8 bytes with TLS_PSK_WITH_AES_128_CCM_8 [RFC6655]), and padding required by the cipher suite. Whether the use of a given mode of DTLS is applicable for a CoAP-based application should be carefully weighed considering the specific cipher suites that may be applicable, whether the session maintenance makes it compatible with application flows, and whether sufficient resources are available on the constrained nodes and for the added network overhead. (For some modes of using DTLS, this specification identifies a mandatory-to-implement cipher suite. This is an implementation requirement to maximize interoperability in those cases where these cipher suites are indeed appropriate. The specific security policies of an application may determine the actual set of cipher suites that can be used.) DTLS is not applicable to group keying (multicast communication); however, it may be a component in a future group key management protocol.

#### 9.1.1. Messaging Layer

The endpoint acting as the CoAP client should also act as the DTLS client. It should initiate a session to the server on the appropriate port. When the DTLS handshake has finished, the client may initiate the first CoAP request. All CoAP messages MUST be sent as DTLS "application data".

The following rules are added for matching an Acknowledgement message or Reset message to a Confirmable message, or a Reset message to a Non-confirmable message: The DTLS session MUST be the same, and the epoch MUST be the same.

A message is the same when it is sent within the same DTLS session and same epoch and has the same Message ID.

Note: When a Confirmable message is retransmitted, a new DTLS sequence_number is used for each attempt, even though the CoAP Message ID stays the same. So a recipient still has to perform deduplication as described in Section 4.5. Retransmissions MUST NOT be performed across epochs.

DTLS connections in RawPublicKey and Certificate mode are set up using mutual authentication so they can remain up and be reused for future message exchanges in either direction. Devices can close a DTLS connection when they need to recover resources, but in general they should keep the connection up for as long as possible. Closing the DTLS connection after every CoAP message exchange is very inefficient.

#### 9.1.2. Request/Response Layer

The following rules are added for matching a response to a request: The DTLS session MUST be the same, and the epoch MUST be the same.

This means the response to a DTLS secured request MUST always be DTLS secured using the same security session and epoch. Any attempt to supply a NoSec response to a DTLS request simply does not match the request and therefore MUST be rejected (unless it does match an unrelated NoSec request).

#### 9.1.3. Endpoint Identity

Devices SHOULD support the Server Name Indication (SNI) to indicate their authority in the SNI HostName field as defined in Section 3 of [RFC6066]. This is needed so that when a host that acts as a virtual server for multiple Authorities receives a new DTLS connection, it knows which keys to use for the DTLS session.

##### 9.1.3.1. Pre-Shared Keys

When forming a connection to a new node, the system selects an appropriate key based on which nodes it is trying to reach and then forms a DTLS session using a PSK (Pre-Shared Key) mode of DTLS. Implementations in these modes MUST support the mandatory-to-implement cipher suite TLS_PSK_WITH_AES_128_CCM_8 as specified in [RFC6655].

Depending on the commissioning model, applications may need to define an application profile for identity hints (as required and detailed in Section 5.2 of [RFC4279]) to enable the use of PSK identity hints.

The security considerations of Section 7 of [RFC4279] apply. In particular, applications should carefully weigh whether or not they need Perfect Forward Secrecy (PFS) and select an appropriate cipher suite (Section 7.1 of [RFC4279]). The entropy of the PSK must be sufficient to mitigate against brute-force and (where the PSK is not chosen randomly but by a human) dictionary attacks (Section 7.2 of [RFC4279]). The cleartext communication of client identities may leak data or compromise privacy (Section 7.3 of [RFC4279]).

#### 9.1.3.2. Raw Public Key Certificates

In this mode, the device has an asymmetric key pair but without an X.509 certificate (called a raw public key); for example, the asymmetric key pair is generated by the manufacturer and installed on the device (see also Section 11.6). A device MAY be configured with multiple raw public keys. The type and length of the raw public key depends on the cipher suite used. Implementations in RawPublicKey mode MUST support the mandatory-to-implement cipher suite TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 as specified in [RFC7251], [RFC5246], and [RFC4492]. The key used MUST be ECDSA capable. The curve secp256r1 MUST be supported [RFC4492]; this curve is equivalent to the NIST P-256 curve. The hash algorithm is SHA-256. Implementations MUST use the Supported Elliptic Curves and Supported Point Formats Extensions [RFC4492]; the uncompressed point format MUST be supported; [RFC6090] can be used as an implementation method. Some guidance relevant to the implementation of this cipher suite can be found in [W3CXMLSEC]. The mechanism for using raw public keys with TLS is specified in [RFC7250].

Implementation Note: Specifically, this means the extensions listed in Figure 14 with at least the values listed will be present in the DTLS handshake.

```
Extension: elliptic_curves
 Type: elliptic_curves (0x000a)
 Length: 4
 Elliptic Curves Length: 2
 Elliptic curves (1 curve)
   Elliptic curve: secp256r1 (0x0017)

Extension: ec_point_formats
 Type: ec_point_formats (0x000b)
 Length: 2
 EC point formats Length: 1
 Elliptic curves point formats (1)
   EC point format: uncompressed (0)

Extension: signature_algorithms
 Type: signature_algorithms (0x000d)
 Length: 4
 Data (4 bytes): 00 02 04 03
   HashAlgorithm: sha256 (4)
   SignatureAlgorithm: ecdsa (3)
```

Figure 14: DTLS Extensions Present for TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8

#### 9.1.3.2.1. Provisioning

The RawPublicKey mode was designed to be easily provisioned in M2M deployments. It is assumed that each device has an appropriate asymmetric public key pair installed. An identifier is calculated by the endpoint from the public key as described in Section 2 of [RFC6920]. All implementations that support checking RawPublicKey identities MUST support at least the sha-256-120 mode (SHA-256 truncated to 120 bits). Implementations SHOULD also support longer length identifiers and MAY support shorter lengths. Note that the shorter lengths provide less security against attacks, and their use is NOT RECOMMENDED.

Depending on how identifiers are given to the system that verifies them, support for URI, binary, and/or human-speakable format [RFC6920] needs to be implemented. All implementations SHOULD support the binary mode, and implementations that have a user interface SHOULD also support the human-speakable format.

During provisioning, the identifier of each node is collected, for example, by reading a barcode on the outside of the device or by obtaining a pre-compiled list of the identifiers. These identifiers are then installed in the corresponding endpoint, for example, an M2M data collection server. The identifier is used for two purposes, to associate the endpoint with further device information and to perform access control. During (initial and ongoing) provisioning, an access control list of identifiers with which the device may start DTLS sessions SHOULD also be installed and maintained.

### 9.1.3.3. X.509 Certificates

Implementations in Certificate Mode MUST support the mandatory-to-implement cipher suite TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 as specified in [RFC7251], [RFC5246], and [RFC4492]. Namely, the certificate includes a SubjectPublicKeyInfo that indicates an algorithm of id-ecPublicKey with namedCurves secp256r1 [RFC5480]; the public key format is uncompressed [RFC5480]; the hash algorithm is SHA-256; if included, the key usage extension indicates digitalSignature. Certificates MUST be signed with ECDSA using secp256r1, and the signature MUST use SHA-256. The key used MUST be ECDSA capable. The curve secp256r1 MUST be supported [RFC4492]; this curve is equivalent to the NIST P-256 curve. The hash algorithm is SHA-256. Implementations MUST use the Supported Elliptic Curves and Supported Point Formats Extensions [RFC4492]; the uncompressed point format MUST be supported; [RFC6090] can be used as an implementation method.

The subject in the certificate would be built out of a long-term unique identifier for the device such as the EUI-64 [EUI64]. The subject could also be based on the Fully Qualified Domain Name (FQDN) that was used as the Host part of the CoAP URI. However, the device's IP address should not typically be used as the subject, as it would change over time. The discovery process used in the system would build up the mapping between IP addresses of the given devices and the subject for each device. Some devices could have more than one subject and would need more than a single certificate.

When a new connection is formed, the certificate from the remote device needs to be verified. If the CoAP node has a source of absolute time, then the node SHOULD check that the validity dates of the certificate are within range. The certificate MUST be validated as appropriate for the security requirements, using functionality equivalent to the algorithm specified in Section 6 of [RFC5280]. If the certificate contains a SubjectAltName, then the authority of the request URI MUST match at least one of the authorities of any CoAP URI found in a field of URI type in the SubjectAltName set. If there is no SubjectAltName in the certificate, then the authority of the request URI MUST match the Common Name (CN) found in the certificate using the matching rules defined in [RFC3280] with the exception that certificates with wildcards are not allowed.

CoRE support for certificate status checking requires further study. As a mapping of the Online Certificate Status Protocol (OCSP) [RFC6960] onto CoAP is not currently defined and OCSP may also not be easily applicable in all environments, an alternative approach may be using the TLS Certificate Status Request extension (Section 8 of [RFC6066]; also known as "OCSP stapling") or preferably the Multiple Certificate Status Extension ([RFC6961]), if available.

If the system has a shared key in addition to the certificate, then a cipher suite that includes the shared key such as TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA [RFC5489] SHOULD be used.

## 10. Cross-Protocol Proxying between CoAP and HTTP

CoAP supports a limited subset of HTTP functionality, and thus cross-protocol proxying to HTTP is straightforward. There might be several reasons for proxying between CoAP and HTTP, for example, when designing a web interface for use over either protocol or when realizing a CoAP-HTTP proxy. Likewise, CoAP could equally be proxied to other protocols such as XMPP [RFC6120] or SIP [RFC3264]; the definition of these mechanisms is out of scope for this specification.

There are two possible directions to access a resource via a forward-proxy:

CoAP-HTTP Proxying: Enables CoAP clients to access resources on HTTP servers through an intermediary. This is initiated by including the Proxy-Uri or Proxy-Scheme Option with an "http" or "https" URI in a CoAP request to a CoAP-HTTP proxy.

HTTP-CoAP Proxying: Enables HTTP clients to access resources on CoAP servers through an intermediary. This is initiated by specifying a "coap" or "coaps" URI in the Request-Line of an HTTP request to an HTTP-CoAP proxy.

Either way, only the request/response model of CoAP is mapped to HTTP. The underlying model of Confirmable or Non-confirmable messages, etc., is invisible and MUST have no effect on a proxy function. The following sections describe the handling of requests to a forward-proxy. Reverse-proxies are not specified, as the proxy function is transparent to the client with the proxy acting as if it were the origin server. However, similar considerations apply to reverse-proxies as to forward-proxies, and there generally will be an expectation that reverse-proxies operate in a similar way forward-proxies would. As an implementation note, HTTP client libraries may make it hard to operate an HTTP-CoAP forward-proxy by not providing a way to put a CoAP URI on the HTTP Request-Line; reverse-proxying may therefore lead to wider applicability of a proxy. A separate specification may define a convention for URIs operating such an HTTP-CoAP reverse-proxy [MAPPING].

### 10.1. CoAP-HTTP Proxying

If a request contains a Proxy-Uri or Proxy-Scheme Option with an 'http' or 'https' URI [RFC2616], then the receiving CoAP endpoint (called "the proxy" henceforth) is requested to perform the operation specified by the request method on the indicated HTTP resource and return the result to the client. (See also Section 5.7 for how the request to the proxy is formulated, including security requirements.)

This section specifies for any CoAP request the CoAP response that the proxy should return to the client. How the proxy actually satisfies the request is an implementation detail, although the typical case is expected to be that the proxy translates and forwards the request to an HTTP origin server.

Since HTTP and CoAP share the basic set of request methods, performing a CoAP request on an HTTP resource is not so different from performing it on a CoAP resource. The meanings of the individual CoAP methods when performed on HTTP resources are explained in the subsections of this section.

If the proxy is unable or unwilling to service a request with an HTTP URI, a 5.05 (Proxying Not Supported) response is returned to the client. If the proxy services the request by interacting with a third party (such as the HTTP origin server) and is unable to obtain a result within a reasonable time frame, a 5.04 (Gateway Timeout) response is returned; if a result can be obtained but is not understood, a 5.02 (Bad Gateway) response is returned.

#### 10.1.1. GET

The GET method requests the proxy to return a representation of the HTTP resource identified by the request URI.

Upon success, a 2.05 (Content) Response Code SHOULD be returned. The payload of the response MUST be a representation of the target HTTP resource, and the Content-Format Option MUST be set accordingly. The response MUST indicate a Max-Age value that is no greater than the remaining time the representation can be considered fresh. If the HTTP entity has an entity-tag, the proxy SHOULD include an ETag Option in the response and process ETag Options in requests as described below.

A client can influence the processing of a GET request by including the following option:

Accept: The request MAY include an Accept Option, identifying the preferred response content-format.

ETag: The request MAY include one or more ETag Options, identifying responses that the client has stored. This requests the proxy to send a 2.03 (Valid) response whenever it would send a 2.05 (Content) response with an entity-tag in the requested set otherwise. Note that CoAP ETags are always strong ETags in the HTTP sense; CoAP does not have the equivalent of HTTP weak ETags, and there is no good way to make use of these in a cross-proxy.

#### 10.1.2. PUT

The PUT method requests the proxy to update or create the HTTP resource identified by the request URI with the enclosed representation.

If a new resource is created at the request URI, a 2.01 (Created) response MUST be returned to the client. If an existing resource is modified, a 2.04 (Changed) response MUST be returned to indicate successful completion of the request.

#### 10.1.3. DELETE

The DELETE method requests the proxy to delete the HTTP resource identified by the request URI at the HTTP origin server.

A 2.02 (Deleted) response MUST be returned to the client upon success or if the resource does not exist at the time of the request.

#### 10.1.4. POST

The POST method requests the proxy to have the representation enclosed in the request be processed by the HTTP origin server. The actual function performed by the POST method is determined by the origin server and dependent on the resource identified by the request URI.

If the action performed by the POST method does not result in a resource that can be identified by a URI, a 2.04 (Changed) response MUST be returned to the client. If a resource has been created on the origin server, a 2.01 (Created) response MUST be returned.

### 10.2. HTTP-CoAP Proxying

If an HTTP request contains a Request-URI with a "coap" or "coaps" URI, then the receiving HTTP endpoint (called "the proxy" henceforth) is requested to perform the operation specified by the request method on the indicated CoAP resource and return the result to the client.

This section specifies for any HTTP request the HTTP response that the proxy should return to the client. Unless otherwise specified, all the statements made are RECOMMENDED behavior; some highly constrained implementations may need to resort to shortcuts. How the proxy actually satisfies the request is an implementation detail, although the typical case is expected to be that the proxy translates and forwards the request to a CoAP origin server. The meanings of the individual HTTP methods when performed on CoAP resources are explained in the subsections of this section.

If the proxy is unable or unwilling to service a request with a CoAP URI, a 501 (Not Implemented) response is returned to the client. If the proxy services the request by interacting with a third party (such as the CoAP origin server) and is unable to obtain a result within a reasonable time frame, a 504 (Gateway Timeout) response is returned; if a result can be obtained but is not understood, a 502 (Bad Gateway) response is returned.

#### 10.2.1. OPTIONS and TRACE

As the OPTIONS and TRACE methods are not supported in CoAP, a 501 (Not Implemented) error MUST be returned to the client.

#### 10.2.2. GET

The GET method requests the proxy to return a representation of the CoAP resource identified by the Request-URI.

Upon success, a 200 (OK) response is returned. The payload of the response MUST be a representation of the target CoAP resource, and the Content-Type and Content-Encoding header fields MUST be set accordingly. The response MUST indicate a max-age directive that indicates a value no greater than the remaining time the representation can be considered fresh. If the CoAP response has an ETag option, the proxy should include an ETag header field in the response.

A client can influence the processing of a GET request by including the following options:

Accept: The most-preferred media type of the HTTP Accept header field in a request is mapped to a CoAP Accept option. HTTP Accept media-type ranges, parameters, and extensions are not supported by the CoAP Accept option. If the proxy cannot send a response that is acceptable according to the combined Accept field value, then the proxy sends a 406 (Not Acceptable) response. The proxy MAY then retry the request with further media types from the HTTP Accept header field.

Conditional GETs: Conditional HTTP GET requests that include an "If-Match" or "If-None-Match" request-header field can be mapped to a corresponding CoAP request. The "If-Modified-Since" and "If-Unmodified-Since" request-header fields are not directly supported by CoAP but are implemented locally by a caching proxy.

#### 10.2.3. HEAD

The HEAD method is identical to GET except that the server MUST NOT return a message-body in the response.

Although there is no direct equivalent of HTTP's HEAD method in CoAP, an HTTP-CoAP proxy responds to HEAD requests for CoAP resources, and the HTTP headers are returned without a message-body.

Implementation Note: An HTTP-CoAP proxy may want to try using a block-wise transfer option [BLOCK] to minimize the amount of data actually transferred, but it needs to be prepared for the case that the origin server does not support block-wise transfers.

#### 10.2.4. POST

The POST method requests the proxy to have the representation enclosed in the request be processed by the CoAP origin server. The actual function performed by the POST method is determined by the origin server and dependent on the resource identified by the request URI.

If the action performed by the POST method does not result in a resource that can be identified by a URI, a 200 (OK) or 204 (No Content) response MUST be returned to the client. If a resource has been created on the origin server, a 201 (Created) response MUST be returned.

If any of the Location-* Options are present in the CoAP response, a Location header field constructed from the values of these options is returned.

#### 10.2.5. PUT

The PUT method requests the proxy to update or create the CoAP resource identified by the Request-URI with the enclosed representation.

If a new resource is created at the Request-URI, a 201 (Created) response is returned to the client. If an existing resource is modified, either the 200 (OK) or 204 (No Content) Response Codes is sent to indicate successful completion of the request.

#### 10.2.6. DELETE

The DELETE method requests the proxy to delete the CoAP resource identified by the Request-URI at the CoAP origin server.

A successful response is 200 (OK) if the response includes an entity describing the status or 204 (No Content) if the action has been enacted but the response does not include an entity.

#### 10.2.7. CONNECT

This method cannot currently be satisfied by an HTTP-CoAP proxy function, as TLS to DTLS tunneling has not yet been specified. For now, a 501 (Not Implemented) error is returned to the client.

## 11. Security Considerations

This section analyzes the possible threats to the protocol. It is meant to inform protocol and application developers about the security limitations of CoAP as described in this document. As CoAP realizes a subset of the features in HTTP/1.1, the security considerations in Section 15 of [RFC2616] are also pertinent to CoAP. This section concentrates on describing limitations specific to CoAP.

### 11.1. Parsing the Protocol and Processing URIs

A network-facing application can exhibit vulnerabilities in its processing logic for incoming packets. Complex parsers are well-known as a likely source of such vulnerabilities, such as the ability to remotely crash a node, or even remotely execute arbitrary code on it. CoAP attempts to narrow the opportunities for introducing such vulnerabilities by reducing parser complexity, by giving the entire range of encodable values a meaning where possible, and by aggressively reducing complexity that is often caused by unnecessary choice between multiple representations that mean the same thing. Much of the URI processing has been moved to the clients, further reducing the opportunities for introducing vulnerabilities into the servers. Even so, the URI processing code in CoAP implementations is likely to be a large source of remaining vulnerabilities and should be implemented with special care. CoAP access control implementations need to ensure they don't introduce vulnerabilities through discrepancies between the code deriving access control decisions from a URI and the code finally serving up the resource addressed by the URI. The most complex parser remaining could be the one for the CoRE Link Format, although this also has been designed with a goal of reduced implementation complexity [RFC6690]. (See also Section 15.2 of [RFC2616].)

### 11.2. Proxying and Caching

As mentioned in Section 15.7 of [RFC2616], proxies are by their very nature men-in-the-middle, breaking any IPsec or DTLS protection that a direct CoAP message exchange might have. They are therefore interesting targets for breaking confidentiality or integrity of CoAP message exchanges. As noted in [RFC2616], they are also interesting targets for breaking availability.

The threat to confidentiality and integrity of request/response data is amplified where proxies also cache. Note that CoAP does not define any of the cache-suppressing Cache-Control options that HTTP/1.1 provides to better protect sensitive data.

For a caching implementation, any access control considerations that would apply to making the request that generated the cache entry also need to be applied to the value in the cache. This is relevant for clients that implement multiple security domains, as well as for proxies that may serve multiple clients. Also, a caching proxy MUST NOT make cached values available to requests that have lesser transport-security properties than those the proxy would require to perform request forwarding in the first place.

Unlike the "coap" scheme, responses to "coaps" identified requests are never "public" and thus MUST NOT be reused for shared caching, unless the cache is able to make equivalent access control decisions to the ones that led to the cached entry. They can, however, be reused in a private cache if the message is cacheable by default in CoAP.

Finally, a proxy that fans out Separate Responses (as opposed to piggybacked Responses) to multiple original requesters may provide additional amplification (see Section 11.3).

### 11.3. Risk of Amplification

CoAP servers generally reply to a request packet with a response packet. This response packet may be significantly larger than the request packet. An attacker might use CoAP nodes to turn a small attack packet into a larger attack packet, an approach known as amplification. There is therefore a danger that CoAP nodes could become implicated in denial-of-service (DoS) attacks by using the amplifying properties of the protocol: an attacker that is attempting to overload a victim but is limited in the amount of traffic it can generate can use amplification to generate a larger amount of traffic.

This is particularly a problem in nodes that enable NoSec access, are accessible from an attacker, and can access potential victims (e.g., on the general Internet), as the UDP protocol provides no way to verify the source address given in the request packet. An attacker need only place the IP address of the victim in the source address of a suitable request packet to generate a larger packet directed at the victim.

As a mitigating factor, many constrained networks will only be able to generate a small amount of traffic, which may make CoAP nodes less attractive for this attack. However, the limited capacity of the constrained network makes the network itself a likely victim of an amplification attack.

Therefore, large amplification factors SHOULD NOT be provided in the response if the request is not authenticated. A CoAP server can reduce the amount of amplification it provides to an attacker by using slicing/blocking modes of CoAP [BLOCK] and offering large resource representations only in relatively small slices. For example, for a 1000-byte resource, a 10-byte request might result in an 80-byte response (with a 64-byte block) instead of a 1016-byte response, considerably reducing the amplification provided.

CoAP also supports the use of multicast IP addresses in requests, an important requirement for M2M. Multicast CoAP requests may be the source of accidental or deliberate DoS attacks, especially over constrained networks. This specification attempts to reduce the amplification effects of multicast requests by limiting when a response is returned. To limit the possibility of malicious use, CoAP servers SHOULD NOT accept multicast requests that can not be authenticated in some way, cryptographically or by some multicast boundary limiting the potential sources. If possible, a CoAP server SHOULD limit the support for multicast requests to the specific resources where the feature is required.

On some general-purpose operating systems providing a POSIX-style API [IEEE1003.1], it is not straightforward to find out whether a packet received was addressed to a multicast address. While many implementations will know whether they have joined a multicast group, this creates a problem for packets addressed to multicast addresses of the form FF0x::1, which are received by every IPv6 node. Implementations SHOULD make use of modern APIs such as IPV6_RECVPKTINFO [RFC3542], if available, to make this determination.

### 11.4. IP Address Spoofing Attacks

Due to the lack of a handshake in UDP, a rogue endpoint that is free to read and write messages carried by the constrained network (i.e., NoSec or PreSharedKey deployments with a nodes/key ratio > 1:1), may easily attack a single endpoint, a group of endpoints, as well as the whole network, e.g., by:

1. spoofing a Reset message in response to a Confirmable message or Non-confirmable message, thus making an endpoint "deaf"; or

2. spoofing an ACK in response to a CON message, thus potentially preventing the sender of the CON message from retransmitting, and drowning out the actual response; or

3. spoofing the entire response with forged payload/options (this has different levels of impact: from single-response disruption, to much bolder attacks on the supporting infrastructure, e.g., poisoning proxy caches, or tricking validation/lookup interfaces in resource directories and, more generally, any component that stores global network state and uses CoAP as the messaging facility to handle setting or updating state is a potential target.); or

4. spoofing a multicast request for a target node; this may result in network congestion/collapse, a DoS attack on the victim, or forced wake-up from sleeping; or

5. spoofing observe messages, etc.

Response spoofing by off-path attackers can be detected and mitigated even without transport layer security by choosing a nontrivial, randomized token in the request (Section 5.3.1). [RFC4086] discusses randomness requirements for security.

In principle, other kinds of spoofing can be detected by CoAP only in case Confirmable message semantics is used, because of unexpected Acknowledgement or Reset messages coming from the deceived endpoint. But this imposes keeping track of the used Message IDs, which is not always possible, and moreover detection becomes available usually after the damage is already done. This kind of attack can be prevented using security modes other than NoSec.

With or without source address spoofing, a client can attempt to overload a server by sending requests, preferably complex ones, to a server; address spoofing makes tracing back, and blocking, this attack harder. Given that the cost of a CON request is small, this attack can easily be executed. Under this attack, a constrained node

with limited total energy available may exhaust that energy much more quickly than planned (battery depletion attack). Also, if the client uses a Confirmable message and the server responds with a Confirmable separate response to a (possibly spoofed) address that does not respond, the server will have to allocate buffer and retransmission logic for each response up to the exhaustion of MAX_TRANSMIT_SPAN, making it more likely that it runs out of resources for processing legitimate traffic. The latter problem can be mitigated somewhat by limiting the rate of responses as discussed in Section 4.7. An attacker could also spoof the address of a legitimate client; this might cause the server, if it uses separate responses, to block legitimate responses to that client because of NSTART=1. All these attacks can be prevented using a security mode other than NoSec, thus leaving only attacks on the security protocol.

### 11.5. Cross-Protocol Attacks

The ability to incite a CoAP endpoint to send packets to a fake source address can be used not only for amplification, but also for cross-protocol attacks against a victim listening to UDP packets at a given address (IP address and port). This would occur as follows:

- The attacker sends a message to a CoAP endpoint with the given address as the fake source address.
- The CoAP endpoint replies with a message to the given source address.
- The victim at the given address receives a UDP packet that it interprets according to the rules of a different protocol.

This may be used to circumvent firewall rules that prevent direct communication from the attacker to the victim but happen to allow communication from the CoAP endpoint (which may also host a valid role in the other protocol) to the victim.

Also, CoAP endpoints may be the victim of a cross-protocol attack generated through an endpoint of another UDP-based protocol such as DNS. In both cases, attacks are possible if the security properties of the endpoints rely on checking IP addresses (and firewalling off direct attacks sent from outside using fake IP addresses). In general, because of their lack of context, UDP-based protocols are relatively easy targets for cross-protocol attacks.

Finally, CoAP URIs transported by other means could be used to incite clients to send messages to endpoints of other protocols.

One mitigation against cross-protocol attacks is strict checking of the syntax of packets received, combined with sufficient difference in syntax. As an example, it might help if it were difficult to incite a DNS server to send a DNS response that would pass the checks of a CoAP endpoint. Unfortunately, the first two bytes of a DNS reply are an ID that can be chosen by the attacker and that map into the interesting part of the CoAP header, and the next two bytes are then interpreted as CoAP's Message ID (i.e., any value is acceptable). The DNS count words may be interpreted as multiple instances of a (nonexistent but elective) CoAP option 0, or possibly as a Token. The echoed query finally may be manufactured by the attacker to achieve a desired effect on the CoAP endpoint; the response added by the server (if any) might then just be interpreted as added payload.

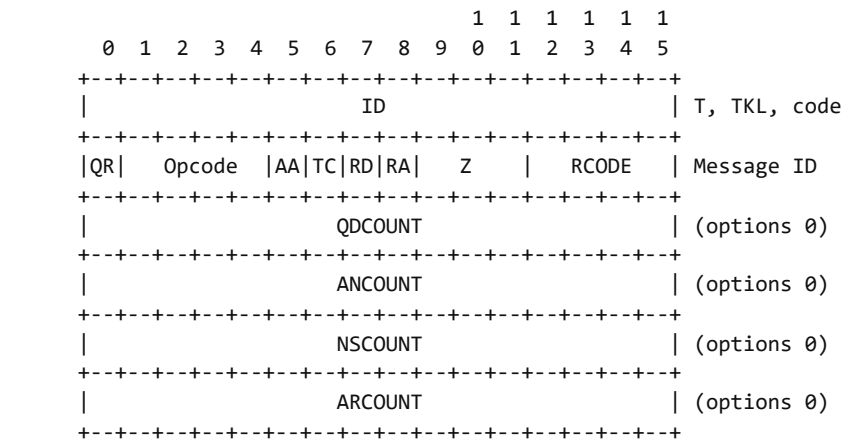



Figure 15: DNS Header ([RFC1035], Section 4.1.1) vs. CoAP Message

In general, for any pair of protocols, one of the protocols can very well have been designed in a way that enables an attacker to cause the generation of replies that look like messages of the other protocol. It is often much harder to ensure or prove the absence of viable attacks than to generate examples that may not yet completely enable an attack but might be further developed by more creative minds. Cross-protocol attacks can therefore only be completely mitigated if endpoints don't authorize actions desired by an attacker just based on trusting the source IP address of a packet. Conversely, a NoSec environment that completely relies on a firewall for CoAP security not only needs to firewall off the CoAP endpoints but also all other endpoints that might be incited to send UDP messages to CoAP endpoints using some other UDP-based protocol.

In addition to the considerations above, the security considerations for DTLS with respect to cross-protocol attacks apply. For example, if the same DTLS security association ("connection") is used to carry data of multiple protocols, DTLS no longer provides protection against cross-protocol attacks between these protocols.

### 11.6. Constrained-Node Considerations

Implementers on constrained nodes often find themselves without a good source of entropy [RFC4086]. If that is the case, the node MUST NOT be used for processes that require good entropy, such as key generation. Instead, keys should be generated externally and added to the device during manufacturing or commissioning.

Due to their low processing power, constrained nodes are particularly susceptible to timing attacks. Special care must be taken in implementation of cryptographic primitives.

Large numbers of constrained nodes will be installed in exposed environments and will have little resistance to tampering, including recovery of keying materials. This needs to be considered when defining the scope of credentials assigned to them. In particular, assigning a shared key to a group of nodes may make any single constrained node a target for subverting the entire group.

## 12. IANA Considerations

### 12.1. CoAP Code Registries

This document defines two sub-registries for the values of the Code field in the CoAP header within the "Constrained RESTful Environments (CoRE) Parameters" registry, hereafter referred to as the "CoRE Parameters" registry.

Values in the two sub-registries are eight-bit values notated as three decimal digits c.dd separated by a period between the first and the second digit; the first digit c is between 0 and 7 and denotes the code class; the second and third digits dd denote a decimal number between 00 and 31 for the detail.

All Code values are assigned by sub-registries according to the following ranges:

0.00 Indicates an Empty message (see Section 4.1).

0.01-0.31 Indicates a request. Values in this range are assigned by the "CoAP Method Codes" sub-registry (see Section 12.1.1).

1.00-1.31 Reserved

2.00-5.31 Indicates a response. Values in this range are assigned by the "CoAP Response Codes" sub-registry (see Section 12.1.2).

6.00-7.31 Reserved

### 12.1.1. Method Codes

The name of the sub-registry is "CoAP Method Codes".

Each entry in the sub-registry must include the Method Code in the range 0.01-0.31, the name of the method, and a reference to the method's documentation.

Initial entries in this sub-registry are as follows:

| Code | Name | Reference |
|---|---|---|
| 0.01 | GET | [RFC7252] |
| 0.02 | POST | [RFC7252] |
| 0.03 | PUT | [RFC7252] |
| 0.04 | DELETE | [RFC7252] |

Table 5: CoAP Method Codes

All other Method Codes are Unassigned.

The IANA policy for future additions to this sub-registry is "IETF Review or IESG Approval" as described in [RFC5226].

The documentation of a Method Code should specify the semantics of a request with that code, including the following properties:

- The Response Codes the method returns in the success case.
- Whether the method is idempotent, safe, or both.

### 12.1.2. Response Codes

The name of the sub-registry is "CoAP Response Codes".

Each entry in the sub-registry must include the Response Code in the range 2.00-5.31, a description of the Response Code, and a reference to the Response Code's documentation.

Initial entries in this sub-registry are as follows:

| Code | Description | Reference |
|---|---|---|
| 2.01 | Created | [RFC7252] |
| 2.02 | Deleted | [RFC7252] |
| 2.03 | Valid | [RFC7252] |
| 2.04 | Changed | [RFC7252] |
| 2.05 | Content | [RFC7252] |
| 4.00 | Bad Request | [RFC7252] |
| 4.01 | Unauthorized | [RFC7252] |
| 4.02 | Bad Option | [RFC7252] |
| 4.03 | Forbidden | [RFC7252] |
| 4.04 | Not Found | [RFC7252] |
| 4.05 | Method Not Allowed | [RFC7252] |
| 4.06 | Not Acceptable | [RFC7252] |
| 4.12 | Precondition Failed | [RFC7252] |
| 4.13 | Request Entity Too Large | [RFC7252] |
| 4.15 | Unsupported Content-Format | [RFC7252] |
| 5.00 | Internal Server Error | [RFC7252] |
| 5.01 | Not Implemented | [RFC7252] |
| 5.02 | Bad Gateway | [RFC7252] |
| 5.03 | Service Unavailable | [RFC7252] |
| 5.04 | Gateway Timeout | [RFC7252] |
| 5.05 | Proxying Not Supported | [RFC7252] |

Table 6: CoAP Response Codes

The Response Codes 3.00-3.31 are Reserved for future use. All other Response Codes are Unassigned.

The IANA policy for future additions to this sub-registry is "IETF Review or IESG Approval" as described in [RFC5226].

The documentation of a Response Code should specify the semantics of a response with that code, including the following properties:

- The methods the Response Code applies to.
- Whether payload is required, optional, or not allowed.
- The semantics of the payload. For example, the payload of a 2.05 (Content) response is a representation of the target resource; the payload in an error response is a human-readable diagnostic payload.
- The format of the payload. For example, the format in a 2.05 (Content) response is indicated by the Content-Format Option; the format of the payload in an error response is always Net-Unicode text.
- Whether the response is cacheable according to the freshness model.
- Whether the response is validatable according to the validation model.
- Whether the response causes a cache to mark responses stored for the request URI as not fresh.

### 12.2. CoAP Option Numbers Registry

This document defines a sub-registry for the Option Numbers used in CoAP options within the "CoRE Parameters" registry. The name of the sub-registry is "CoAP Option Numbers".

Each entry in the sub-registry must include the Option Number, the name of the option, and a reference to the option's documentation.

Initial entries in this sub-registry are as follows:

| Number | Name | Reference |
|---|---|---|
| 0 | (Reserved) | [RFC7252] |
| 1 | If-Match | [RFC7252] |
| 3 | Uri-Host | [RFC7252] |
| 4 | ETag | [RFC7252] |
| 5 | If-None-Match | [RFC7252] |
| 7 | Uri-Port | [RFC7252] |
| 8 | Location-Path | [RFC7252] |
| 11 | Uri-Path | [RFC7252] |
| 12 | Content-Format | [RFC7252] |
| 14 | Max-Age | [RFC7252] |
| 15 | Uri-Query | [RFC7252] |
| 17 | Accept | [RFC7252] |
| 20 | Location-Query | [RFC7252] |
| 35 | Proxy-Uri | [RFC7252] |
| 39 | Proxy-Scheme | [RFC7252] |
| 60 | Size1 | [RFC7252] |
| 128 | (Reserved) | [RFC7252] |
| 132 | (Reserved) | [RFC7252] |
| 136 | (Reserved) | [RFC7252] |
| 140 | (Reserved) | [RFC7252] |

Table 7: CoAP Option Numbers

The IANA policy for future additions to this sub-registry is split into three tiers as follows. The range of 0..255 is reserved for options defined by the IETF (IETF Review or IESG Approval). The range of 256..2047 is reserved for commonly used options with public specifications (Specification Required). The range of 2048..64999 is for all other options including private or vendor-specific ones, which undergo a Designated Expert review to help ensure that the option semantics are defined correctly. The option numbers between 65000 and 65535 inclusive are reserved for experiments. They are not meant for vendor-specific use of any kind and MUST NOT be used in operational deployments.

| Range | Registration Procedures |
|---|---|
| 0-255 | IETF Review or IESG Approval |
| 256-2047 | Specification Required |
| 2048-64999 | Expert Review |
| 65000-65535 | Experimental use (no operational use) |

Table 8: CoAP Option Numbers: Registration Procedures

The documentation of an Option Number should specify the semantics of an option with that number, including the following properties:

- The meaning of the option in a request.
- The meaning of the option in a response.
- Whether the option is critical or elective, as determined by the Option Number.
- Whether the option is Safe-to-Forward, and, if yes, whether it is part of the Cache-Key, as determined by the Option Number (see Section 5.4.2).
- The format and length of the option's value.
- Whether the option must occur at most once or whether it can occur multiple times.
- The default value, if any. For a critical option with a default value, a discussion on how the default value enables processing by implementations that do not support the critical option (Section 5.4.4).

### 12.3. CoAP Content-Formats Registry

Internet media types are identified by a string, such as "application/xml" [RFC2046]. In order to minimize the overhead of using these media types to indicate the format of payloads, this document defines a sub-registry for a subset of Internet media types to be used in CoAP and assigns each, in combination with a content-coding, a numeric identifier. The name of the sub-registry is "CoAP Content-Formats", within the "CoRE Parameters" registry.

Each entry in the sub-registry must include the media type registered with IANA, the numeric identifier in the range 0-65535 to be used for that media type in CoAP, the content-coding associated with this identifier, and a reference to a document describing what a payload with that media type means semantically.

CoAP does not include a separate way to convey content-encoding information with a request or response, and for that reason the content-encoding is also specified for each identifier (if any). If multiple content-encodings will be used with a media type, then a separate Content-Format identifier for each is to be registered. Similarly, other parameters related to an Internet media type, such as level, can be defined for a CoAP Content-Format entry.

Initial entries in this sub-registry are as follows:

| Media type | Encoding | ID | Reference |
|---|---|---|---|
| text/plain; charset=utf-8 | - | 0 | [RFC2046] [RFC3676] [RFC5147] |
| application/link-format | - | 40 | [RFC6690] |
| application/xml | - | 41 | [RFC3023] |
| application/octet-stream | - | 42 | [RFC2045] [RFC2046] |
| application/exi | - | 47 | [REC-exi-20140211] |
| application/json | - | 50 | [RFC7159] |

Table 9: CoAP Content-Formats

The identifiers between 65000 and 65535 inclusive are reserved for experiments. They are not meant for vendor-specific use of any kind and MUST NOT be used in operational deployments. The identifiers between 256 and 9999 are reserved for future use in IETF specifications (IETF Review or IESG Approval). All other identifiers are Unassigned.

Because the namespace of single-byte identifiers is so small, the IANA policy for future additions in the range 0-255 inclusive to the sub-registry is "Expert Review" as described in [RFC5226]. The IANA policy for additions in the range 10000-64999 inclusive is "First Come First Served" as described in [RFC5226]. This is summarized in the following table.

| Range | Registration Procedures |
|---|---|
| 0-255 | Expert Review |
| 256-9999 | IETF Review or IESG Approval |
| 10000-64999 | First Come First Served |
| 65000-65535 | Experimental use (no operational use) |

Table 10: CoAP Content-Formats: Registration Procedures

In machine-to-machine applications, it is not expected that generic Internet media types such as text/plain, application/xml or application/octet-stream are useful for real applications in the long term. It is recommended that M2M applications making use of CoAP request new Internet media types from IANA indicating semantic information about how to create or parse a payload. For example, a Smart Energy application payload carried as XML might request a more specific type like application/se+xml or application/se-exi.

### 12.4. URI Scheme Registration

This document contains the request for the registration of the Uniform Resource Identifier (URI) scheme "coap". The registration request complies with [RFC4395].

URI scheme name.
coap

Status.
Permanent.

URI scheme syntax.
Defined in Section 6.1 of [RFC7252].

URI scheme semantics.
The "coap" URI scheme provides a way to identify resources that are potentially accessible over the Constrained Application Protocol (CoAP). The resources can be located by contacting the governing CoAP server and operated on by sending CoAP requests to the server. This scheme can thus be compared to the "http" URI scheme [RFC2616]. See Section 6 of [RFC7252] for the details of operation.

Encoding considerations.
The scheme encoding conforms to the encoding rules established for URIs in [RFC3986], i.e., internationalized and reserved characters are expressed using UTF-8-based percent-encoding.

   Applications/protocols that use this URI scheme name.
      The scheme is used by CoAP endpoints to access CoAP resources.

   Interoperability considerations.
      None.

   Security considerations.
      See Section 11.1 of [RFC7252].

   Contact.
      IETF Chair <chair@ietf.org>

   Author/Change controller.
      IESG <iesg@ietf.org>

   References.
      [RFC7252]

### 12.5.  Secure URI Scheme Registration

   This document contains the request for the registration of the
   Uniform Resource Identifier (URI) scheme "coaps".  The registration
   request complies with [RFC4395].

   URI scheme name.
      coaps

   Status.
      Permanent.

   URI scheme syntax.
      Defined in Section 6.2 of [RFC7252].

   URI scheme semantics.
      The "coaps" URI scheme provides a way to identify resources that
      are potentially accessible over the Constrained Application
      Protocol (CoAP) using Datagram Transport Layer Security (DTLS) for
      transport security.  The resources can be located by contacting
      the governing CoAP server and operated on by sending CoAP requests
      to the server.  This scheme can thus be compared to the "https"
      URI scheme [RFC2616].  See Section 6 of [RFC7252] for the details
      of operation.

   Encoding considerations.
      The scheme encoding conforms to the encoding rules established for
      URIs in [RFC3986], i.e., internationalized and reserved characters
      are expressed using UTF-8-based percent-encoding.

Applications/protocols that use this URI scheme name.
The scheme is used by CoAP endpoints to access CoAP resources using DTLS.

Interoperability considerations.
None.

Security considerations.
See Section 11.1 of [RFC7252].

Contact.
IETF Chair <chair@ietf.org>

Author/Change controller.
IESG <iesg@ietf.org>

References.
[RFC7252]

### 12.6. Service Name and Port Number Registration

One of the functions of CoAP is resource discovery: a CoAP client can ask a CoAP server about the resources offered by it (see Section 7). To enable resource discovery just based on the knowledge of an IP address, the CoAP port for resource discovery needs to be standardized.

IANA has assigned the port number 5683 and the service name "coap", in accordance with [RFC6335].

Besides unicast, CoAP can be used with both multicast and anycast.

Service Name.
coap

Transport Protocol.
udp

Assignee.
IESG <iesg@ietf.org>

Contact.
IETF Chair <chair@ietf.org>

Description.
Constrained Application Protocol (CoAP)

Reference.
[RFC7252]

Port Number.
5683

### 12.7. Secure Service Name and Port Number Registration

CoAP resource discovery may also be provided using the DTLS-secured CoAP "coaps" scheme. Thus, the CoAP port for secure resource discovery needs to be standardized.

IANA has assigned the port number 5684 and the service name "coaps", in accordance with [RFC6335].

Besides unicast, DTLS-secured CoAP can be used with anycast.

Service Name.
coaps

Transport Protocol.
udp

Assignee.
IESG <iesg@ietf.org>

Contact.
IETF Chair <chair@ietf.org>

Description.
DTLS-secured CoAP

Reference.
[RFC7252]

Port Number.
5684

### 12.8. Multicast Address Registration

Section 8, "Multicast CoAP", defines the use of multicast. IANA has assigned the following multicast addresses for use by CoAP nodes:

IPv4 -- "All CoAP Nodes" address 224.0.1.187, from the "IPv4 Multicast Address Space Registry". As the address is used for discovery that may span beyond a single network, it has come from the Internetwork Control Block (224.0.1.x, RFC 5771).

IPv6 -- "All CoAP Nodes" address FF0X::FD, from the "IPv6 Multicast Address Space Registry", in the "Variable Scope Multicast Addresses" space (RFC 3307). Note that there is a distinct multicast address for each scope that interested CoAP nodes should listen to; CoAP needs the Link-Local and Site-Local scopes only.

## 13. Acknowledgements

Brian Frank was a contributor to and coauthor of early versions of this specification.

Special thanks to Peter Bigot, Esko Dijk, and Cullen Jennings for substantial contributions to the ideas and text in the document, along with countless detailed reviews and discussions.

Thanks to Floris Van den Abeele, Anthony Baire, Ed Beroset, Berta Carballido, Angelo P. Castellani, Gilbert Clark, Robert Cragie, Pierre David, Esko Dijk, Lisa Dusseault, Mehmet Ersue, Thomas Fossati, Tobias Gondrom, Bert Greevenbosch, Tom Herbst, Jeroen Hoebeke, Richard Kelsey, Sye Loong Keoh, Ari Keranen, Matthias Kovatsch, Avi Lior, Stephan Lohse, Salvatore Loreto, Kerry Lynn, Andrew McGregor, Alexey Melnikov, Guido Moritz, Petri Mutka, Colin O'Flynn, Charles Palmer, Adriano Pezzuto, Thomas Poetsch, Robert Quattlebaum, Akbar Rahman, Eric Rescorla, Dan Romascanu, David Ryan, Peter Saint-Andre, Szymon Sasin, Michael Scharf, Dale Seed, Robby Simpson, Peter van der Stok, Michael Stuber, Linyi Tian, Gilman Tolle, Matthieu Vial, Maciej Wasilak, Fan Xianyou, and Alper Yegin for helpful comments and discussions that have shaped the document. Special thanks also to the responsible IETF area director at the time of completion, Barry Leiba, and the IESG reviewers, Adrian Farrel, Martin Stiemerling, Pete Resnick, Richard Barnes, Sean Turner, Spencer Dawkins, Stephen Farrell, and Ted Lemon, who contributed in-depth reviews.

Some of the text has been borrowed from the working documents of the IETF HTTPBIS working group.

# 14. References

## 14.1. Normative References

[RFC0768] Postel, J., "User Datagram Protocol", STD 6, RFC 768, August 1980.

[RFC2045] Freed, N. and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies", RFC 2045, November 1996.

[RFC2046] Freed, N. and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part Two: Media Types", RFC 2046, November 1996.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997.

[RFC2616] Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., and T. Berners-Lee, "Hypertext Transfer Protocol -- HTTP/1.1", RFC 2616, June 1999.

[RFC3023] Murata, M., St. Laurent, S., and D. Kohn, "XML Media Types", RFC 3023, January 2001.

[RFC3629] Yergeau, F., "UTF-8, a transformation format of ISO 10646", STD 63, RFC 3629, November 2003.

[RFC3676] Gellens, R., "The Text/Plain Format and DelSp Parameters", RFC 3676, February 2004.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, January 2005.

[RFC4279] Eronen, P. and H. Tschofenig, "Pre-Shared Key Ciphersuites for Transport Layer Security (TLS)", RFC 4279, December 2005.

[RFC4395] Hansen, T., Hardie, T., and L. Masinter, "Guidelines and Registration Procedures for New URI Schemes", BCP 35, RFC 4395, February 2006.

[RFC5147] Wilde, E. and M. Duerst, "URI Fragment Identifiers for the text/plain Media Type", RFC 5147, April 2008.

[RFC5198] Klensin, J. and M. Padlipsky, "Unicode Format for Network Interchange", RFC 5198, March 2008.

[RFC5226] Narten, T. and H. Alvestrand, "Guidelines for Writing an IANA Considerations Section in RFCs", BCP 26, RFC 5226, May 2008.

[RFC5234] Crocker, D. and P. Overell, "Augmented BNF for Syntax Specifications: ABNF", STD 68, RFC 5234, January 2008.

[RFC5246] Dierks, T. and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2", RFC 5246, August 2008.

[RFC5280] Cooper, D., Santesson, S., Farrell, S., Boeyen, S., Housley, R., and W. Polk, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 5280, May 2008.

[RFC5480] Turner, S., Brown, D., Yiu, K., Housley, R., and T. Polk, "Elliptic Curve Cryptography Subject Public Key Information", RFC 5480, March 2009.

[RFC5785] Nottingham, M. and E. Hammer-Lahav, "Defining Well-Known Uniform Resource Identifiers (URIs)", RFC 5785, April 2010.

[RFC5952] Kawamura, S. and M. Kawashima, "A Recommendation for IPv6 Address Text Representation", RFC 5952, August 2010.

[RFC5988] Nottingham, M., "Web Linking", RFC 5988, October 2010.

[RFC6066] Eastlake, D., "Transport Layer Security (TLS) Extensions: Extension Definitions", RFC 6066, January 2011.

[RFC6347] Rescorla, E. and N. Modadugu, "Datagram Transport Layer Security Version 1.2", RFC 6347, January 2012.

[RFC6690] Shelby, Z., "Constrained RESTful Environments (CoRE) Link Format", RFC 6690, August 2012.

[RFC6920] Farrell, S., Kutscher, D., Dannewitz, C., Ohlman, B., Keranen, A., and P. Hallam-Baker, "Naming Things with Hashes", RFC 6920, April 2013.

[RFC7250] Wouters, P., Tschofenig, H., Gilmore, J., Weiler, S., and T. Kivinen, "Using Raw Public Keys in Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)", RFC 7250, June 2014.

[RFC7251] McGrew, D., Bailey, D., Campagna, M., and R. Dugal, "AES-CCM Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS)", RFC 7251, June 2014.

### 14.2. Informative References

[BLOCK] Bormann, C. and Z. Shelby, "Blockwise transfers in CoAP", Work in Progress, October 2013.

[CoAP-MISC] Bormann, C. and K. Hartke, "Miscellaneous additions to CoAP", Work in Progress, December 2013.

[EUI64] IEEE Standards Association, "Guidelines for 64-bit Global Identifier (EUI-64 (TM))", Registration Authority Tutorials, April 2010, <http://standards.ieee.org/regauth/oui/tutorials/EUI64.html>.

[GROUPCOMM] Rahman, A. and E. Dijk, "Group Communication for CoAP", Work in Progress, December 2013.

[HHGTTG] Adams, D., "The Hitchhiker's Guide to the Galaxy", Pan Books ISBN 3320258648, 1979.

[IEEE1003.1] IEEE and The Open Group, "Portable Operating System Interface (POSIX)", The Open Group Base Specifications Issue 7, IEEE 1003.1, 2013 Edition, <http://pubs.opengroup.org/onlinepubs/9699919799/>.

[IPsec-CoAP] Bormann, C., "Using CoAP with IPsec", Work in Progress, December 2012.

[MAPPING] Castellani, A., Loreto, S., Rahman, A., Fossati, T., and E. Dijk, "Guidelines for HTTP-CoAP Mapping Implementations", Work in Progress, February 2014.

[OBSERVE] Hartke, K., "Observing Resources in CoAP", Work in Progress, April 2014.

[REC-exi-20140211] Schneider, J., Kamiya, T., Peintner, D., and R. Kyusakov, "Efficient XML Interchange (EXI) Format 1.0 (Second Edition)", W3C Recommendation REC-exi-20140211, February 2014, <http://www.w3.org/TR/2014/REC-exi-20140211/>.

[REST] Fielding, R., "Architectural Styles and the Design of Network-based Software Architectures", Ph.D. Dissertation, University of California, Irvine, 2000, <http://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf>.

[RFC0020] Cerf, V., "ASCII format for network interchange", RFC 20, October 1969.

[RFC0791] Postel, J., "Internet Protocol", STD 5, RFC 791, September 1981.

[RFC0792] Postel, J., "Internet Control Message Protocol", STD 5, RFC 792, September 1981.

[RFC0793] Postel, J., "Transmission Control Protocol", STD 7, RFC 793, September 1981.

[RFC1035] Mockapetris, P., "Domain names - implementation and specification", STD 13, RFC 1035, November 1987.

[RFC3264] Rosenberg, J. and H. Schulzrinne, "An Offer/Answer Model with Session Description Protocol (SDP)", RFC 3264, June 2002.

[RFC3280] Housley, R., Polk, W., Ford, W., and D. Solo, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 3280, April 2002.

[RFC3542] Stevens, W., Thomas, M., Nordmark, E., and T. Jinmei, "Advanced Sockets Application Program Interface (API) for IPv6", RFC 3542, May 2003.

[RFC3828] Larzon, L-A., Degermark, M., Pink, S., Jonsson, L-E., and G. Fairhurst, "The Lightweight User Datagram Protocol (UDP-Lite)", RFC 3828, July 2004.

[RFC4086] Eastlake, D., Schiller, J., and S. Crocker, "Randomness Requirements for Security", BCP 106, RFC 4086, June 2005.

[RFC4443] Conta, A., Deering, S., and M. Gupta, "Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6) Specification", RFC 4443, March 2006.

[RFC4492] Blake-Wilson, S., Bolyard, N., Gupta, V., Hawk, C., and B. Moeller, "Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS)", RFC 4492, May 2006.

[RFC4821] Mathis, M. and J. Heffner, "Packetization Layer Path MTU Discovery", RFC 4821, March 2007.

[RFC4944] Montenegro, G., Kushalnagar, N., Hui, J., and D. Culler, "Transmission of IPv6 Packets over IEEE 802.15.4 Networks", RFC 4944, September 2007.

[RFC5405] Eggert, L. and G. Fairhurst, "Unicast UDP Usage Guidelines for Application Designers", BCP 145, RFC 5405, November 2008.

[RFC5489] Badra, M. and I. Hajjeh, "ECDHE_PSK Cipher Suites for Transport Layer Security (TLS)", RFC 5489, March 2009.

[RFC6090] McGrew, D., Igoe, K., and M. Salter, "Fundamental Elliptic Curve Cryptography Algorithms", RFC 6090, February 2011.

[RFC6120] Saint-Andre, P., "Extensible Messaging and Presence Protocol (XMPP): Core", RFC 6120, March 2011.

[RFC6282] Hui, J. and P. Thubert, "Compression Format for IPv6 Datagrams over IEEE 802.15.4-Based Networks", RFC 6282, September 2011.

[RFC6335] Cotton, M., Eggert, L., Touch, J., Westerlund, M., and S. Cheshire, "Internet Assigned Numbers Authority (IANA) Procedures for the Management of the Service Name and Transport Protocol Port Number Registry", BCP 165, RFC 6335, August 2011.

[RFC6655] McGrew, D. and D. Bailey, "AES-CCM Cipher Suites for Transport Layer Security (TLS)", RFC 6655, July 2012.

[RFC6936] Fairhurst, G. and M. Westerlund, "Applicability Statement for the Use of IPv6 UDP Datagrams with Zero Checksums", RFC 6936, April 2013.

[RFC6960] Santesson, S., Myers, M., Ankney, R., Malpani, A., Galperin, S., and C. Adams, "X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP", RFC 6960, June 2013.

[RFC6961] Pettersen, Y., "The Transport Layer Security (TLS) Multiple Certificate Status Request Extension", RFC 6961, June 2013.

[RFC7159] Bray, T., "The JavaScript Object Notation (JSON) Data Interchange Format", RFC 7159, March 2014.

[RFC7228] Bormann, C., Ersue, M., and A. Keranen, "Terminology for Constrained-Node Networks", RFC 7228, May 2014.

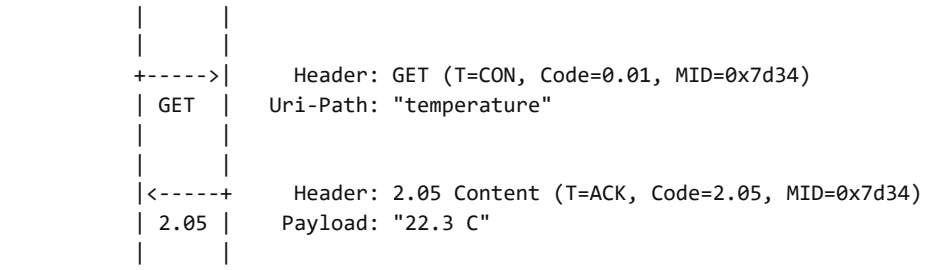
[RTO-CONSIDER]
Allman, M., "Retransmission Timeout Considerations", Work in Progress, May 2012.

[W3CXMLSEC]
Wenning, R., "Report of the XML Security PAG", W3C XML Security PAG, October 2012, <http://www.w3.org/2011/xmlsec-pag/pagreport.html>.

## Appendix A.  Examples

This section gives a number of short examples with message flows for GET requests. These examples demonstrate the basic operation, the operation in the presence of retransmissions, and multicast.

Figure 16 shows a basic GET request causing a piggybacked response: The client sends a Confirmable GET request for the resource coap://server/temperature to the server with a Message ID of 0x7d34. The request includes one Uri-Path Option (Delta 0 + 11 = 11, Length 11, Value "temperature"); the Token is left empty. This request is a total of 16 bytes long. A 2.05 (Content) response is returned in the Acknowledgement message that acknowledges the Confirmable request, echoing both the Message ID 0x7d34 and the empty Token value. The response includes a Payload of "22.3 C" and is 11 bytes long.



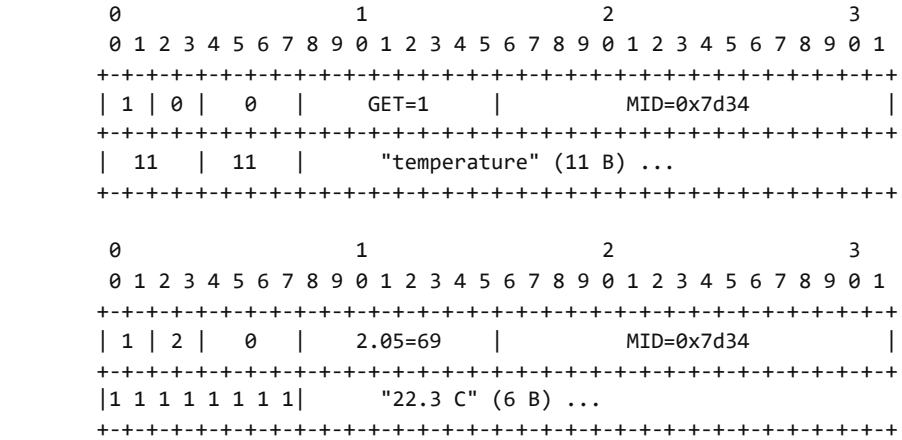
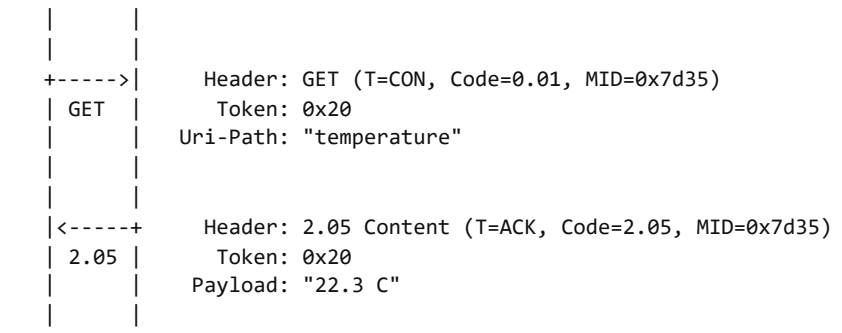
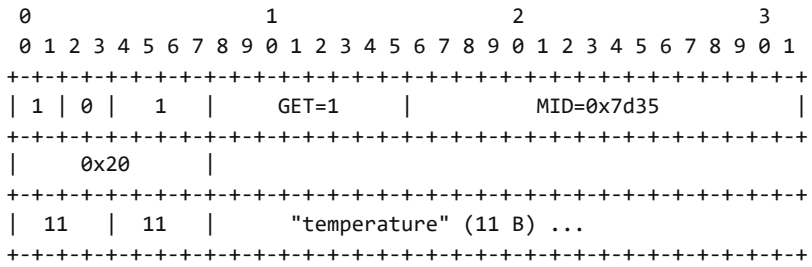




Figure 16: Confirmable Request; Piggybacked Response

Figure 17 shows a similar example, but with the inclusion of an non-empty Token (Value 0x20) in the request and the response, increasing the sizes to 17 and 12 bytes, respectively.



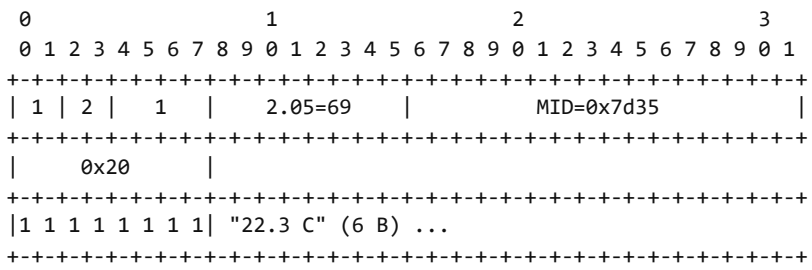




Figure 17: Confirmable Request; Piggybacked Response

In Figure 18, the Confirmable GET request is lost. After ACK_TIMEOUT seconds, the client retransmits the request, resulting in a piggybacked response as in the previous example.

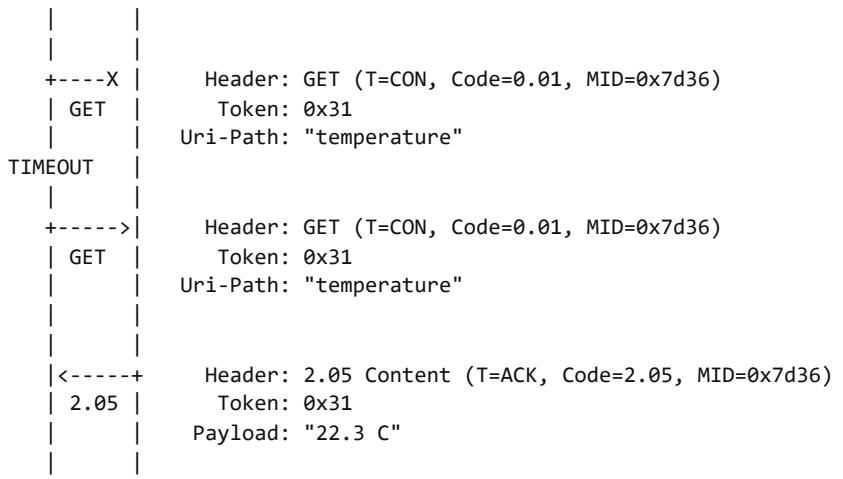



Figure 18: Confirmable Request (Retransmitted); Piggybacked Response

In Figure 19, the first Acknowledgement message from the server to the client is lost. After ACK_TIMEOUT seconds, the client retransmits the request.

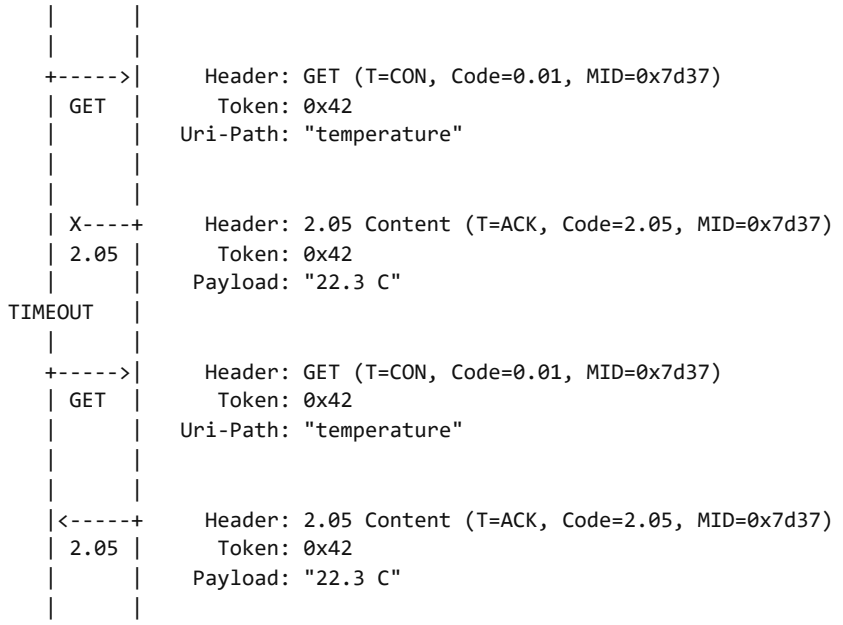



Figure 19: Confirmable Request; Piggybacked Response (Retransmitted)

In Figure 20, the server acknowledges the Confirmable request and sends a 2.05 (Content) response separately in a Confirmable message. Note that the Acknowledgement message and the Confirmable response do not necessarily arrive in the same order as they were sent. The client acknowledges the Confirmable response.

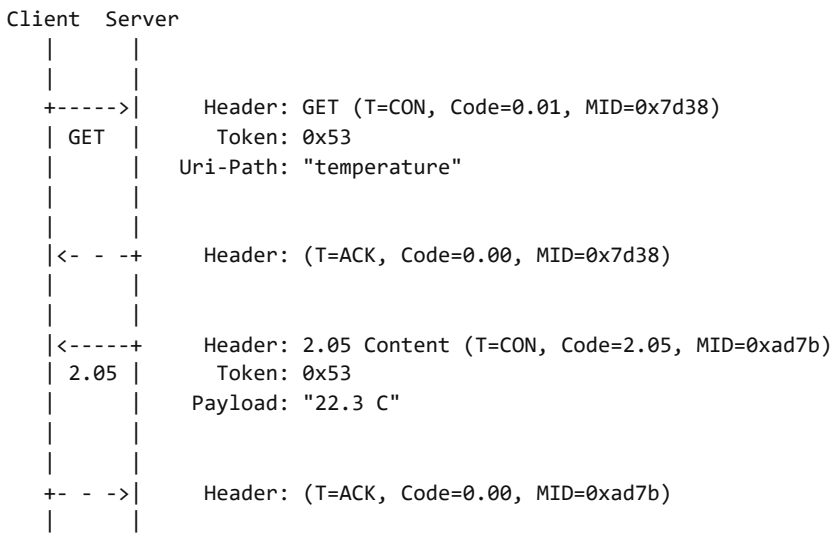



Figure 20: Confirmable Request; Separate Response

Figure 21 shows an example where the client loses its state (e.g., crashes and is rebooted) right after sending a Confirmable request, so the separate response arriving some time later comes unexpected. In this case, the client rejects the Confirmable response with a Reset message. Note that the unexpected ACK is silently ignored.

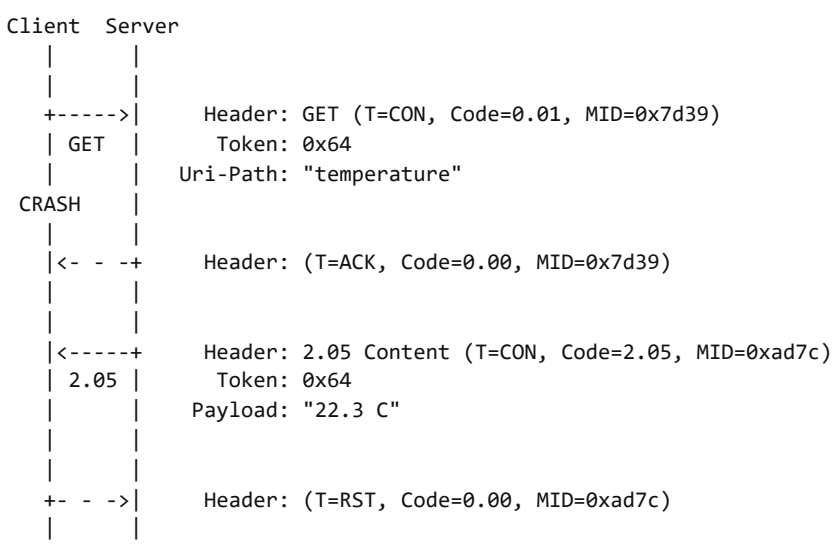



Figure 21: Confirmable Request; Separate Response (Unexpected)

Figure 22 shows a basic GET request where the request and the response are Non-confirmable, so both may be lost without notice.

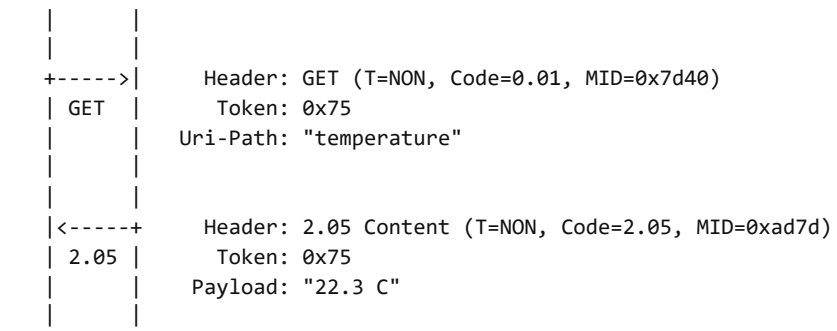



Figure 22: Non-confirmable Request; Non-confirmable Response

In Figure 23, the client sends a Non-confirmable GET request to a multicast address: all nodes in link-local scope. There are 3 servers on the link: A, B and C. Servers A and B have a matching resource, therefore they send back a Non-confirmable 2.05 (Content) response. The response sent by B is lost. C does not have matching response, therefore it sends a Non-confirmable 4.04 (Not Found) response.

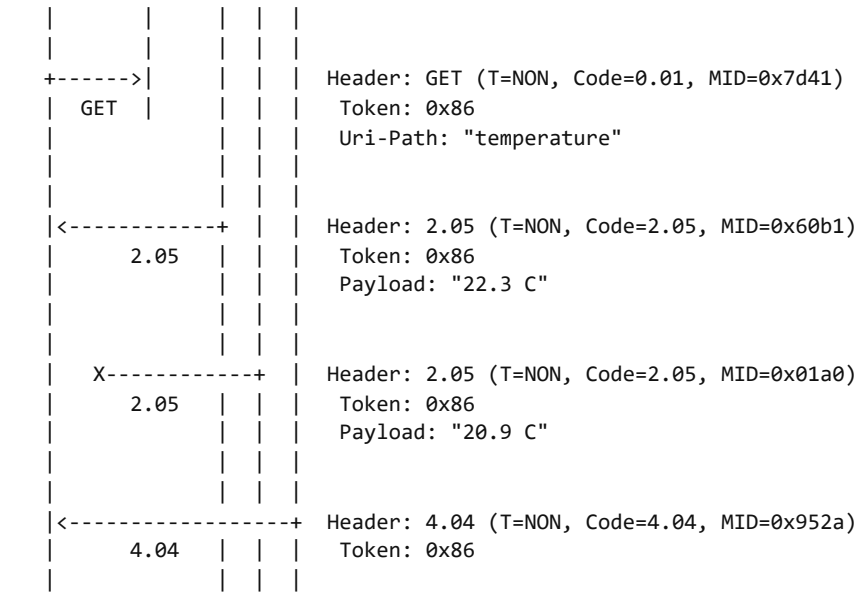



Figure 23: Non-confirmable Request (Multicast); Non-confirmable Response

## Appendix B. URI Examples

The following examples demonstrate different sets of Uri options, and the result after constructing an URI from them. In addition to the options, Section 6.5 refers to the destination IP address and port, but not all paths of the algorithm cause the destination IP address and port to be included in the URI.

- Input:

  Destination IP Address = [2001:db8::2:1]
  Destination UDP Port = 5683

Output:

coap://[2001:db8::2:1]/

- Input:

  Destination IP Address = [2001:db8::2:1]
  Destination UDP Port = 5683
  Uri-Host = "example.net"

  Output:

  coap://example.net/

- Input:

  Destination IP Address = [2001:db8::2:1]
  Destination UDP Port = 5683
  Uri-Host = "example.net"
  Uri-Path = ".well-known"
  Uri-Path = "core"

  Output:

  coap://example.net/.well-known/core

- Input:

  Destination IP Address = [2001:db8::2:1]
  Destination UDP Port = 5683
  Uri-Host = "xn--18j4d.example"
  Uri-Path = the string composed of the Unicode characters U+3053
  U+3093 U+306b U+3061 U+306f, usually represented in UTF-8 as
  E38193E38293E381ABE381A1E381AF hexadecimal

  Output:

  coap://xn--18j4d.example/
  %E3%81%93%E3%82%93%E3%81%AB%E3%81%A1%E3%81%AF

  (The line break has been inserted for readability; it is not
  part of the URI.)

o Input:

```
Destination IP Address = 198.51.100.1
Destination UDP Port = 61616
Uri-Path = ""
Uri-Path = "/"
Uri-Path = ""
Uri-Path = ""
Uri-Query = "//"
Uri-Query = "?&"
```

Output:

```
coap://198.51.100.1:61616//%2F//?%2F%2F&?%26
```

## Authors' Addresses

Zach Shelby
ARM
150 Rose Orchard
San Jose, CA 95134
USA

Phone: +1-408-203-9434
EMail: zach.shelby@arm.com

Klaus Hartke
Universitaet Bremen TZI
Postfach 330440
Bremen D-28359
Germany

Phone: +49-421-218-63905
EMail: hartke@tzi.org

Carsten Bormann
Universitaet Bremen TZI
Postfach 330440
Bremen D-28359
Germany

Phone: +49-421-218-63921
EMail: cabo@tzi.org